

ROS2 MAKER / FREE SAMPLE

ROS2 입문

Ubuntu 24.04·Jazzy·Gazebo Harmonic과

AI로 시작하는 로봇 메이커 가이드

최수길 지음 · 마담

1~3장 무료 공개 | SAMPLE

1장 ROS2로 무엇을 만들 수 있는가

2장 Windows·WSL2·Ubuntu의 관계

3장 Ubuntu 24.04와 ROS2 Jazzy 설치

설치 환경을 이해하고, 두 Node의 실제 통신까지 확인합니다.

B5 컬러 PDF의 9~65쪽을 발췌했습니다.

본문 57쪽 · 원본 쪽 번호 유지 · 전체 도서가 아닌 체험판

<https://github.com/freshmea/ros2-maker-guide>

Copyright © 2026 choi su gil. 무료 열람용 · 판매 금지

본문·이미지는 코드의 Apache-2.0 라이선스에 포함되지 않습니다.



ROS2로 무엇을 만들 수 있는가

01
CHAPTER

메이커 활동이나 기초 프로젝트는 아두이노 같은 보드에 센서와 모터를 연결하고, 버튼을 누르면 LED가 켜지는 코드를 작성하는 데서 시작하는 경우가 많다. 이러한 실습으로 기본 원리를 익힐 수 있지만, 기능이 늘어나면 보드의 성능과 프로그램 구조를 함께 고민하게 된다. 여러 장치와 프로그램을 연결하는 방법을 찾아 보면 ROS2를 만나게 된다.

학습 목표

- 로봇 소프트웨어가 일반적인 단일 프로그램과 다른 이유는 무엇인가?
- ROS2는 운영체제인가, 프로그래밍 언어인가, 아니면 개발 도구인가?
- Node와 Topic은 로봇 시스템에서 어떤 역할을 하는가?
- 실제 로봇이 없어도 ROS2를 배울 수 있는 이유는 무엇인가?
- 이 책을 따라가며 어떤 결과물을 만들게 되는가?

이 장의 학습 지도

01 센서와 데이터

02 Node의 협력

03 로봇 시스템

완료 기준

01 로봇 기능 하나를 센서 입력·판단·동작으로 나누고, 각 기능을 맡을 Node와 데이터 흐름을 그린다.

02 Topic·Service·Action에 알맞은 로봇 사례를 하나씩 들고 선택 이유를 설명한다.

03 Turtlesim·RViz2·Gazebo의 역할을 구분하고, 실제 로봇 없이 확인할 수 있는 결과를 적는다.

04 이 책의 기준 환경과 만들고 싶은 결과물을 기록하고, AI에게 질문할 때 환경과 확인 조건을 함께 쓴다.

개념을 이해하고 · 직접 실행하고 · 결과로 확인하기

ROS2를 처음 배우려면 무엇부터 시작해야 할지 막막할 수 있다. 이 책의 목표는 ROS2의 구조를 이해하고 로봇을 움직이는 프로그램을 만드는 것이다. 하드웨어를 직접 제작하기 어렵더라도 시뮬레이션에서 로봇을 움직이는 것부터 시작해 보자. 작은 성공을 쌓으며 실제 로봇으로 확장할 기반을 마련할 수 있다.

로봇을 만들려면 바퀴를 돌리는 코드, 카메라 영상을 읽는 코드, 장애물을 피하는 코드가 모두 필요해 보인다. 센서와 모터를 연결하는 것만으로도 복잡한데, 여러 프로그램이 동시에 동작하며 데이터를 주고받아야 한다.

ROS2는 이 복잡한 일을 작은 기능으로 나누고 다시 연결하도록 돕는 도구다. 이 책은 명령을 외우기보다 작은 프로그램이 서로 대화하고 협력하여 하나의 로봇을 만드는 원리에 초점을 맞춘다. AI를 활용해 문제를 해결하는 방법도 함께 다룬다.



그림 1-1. ROS2를 사용하면 센서, 이동 로봇, 로봇 팔 같은 서로 다른 기능을 하나의 시스템으로 연결할 수 있다.

이 책이 로봇을 만드는 작은 출발점이 되기를 바란다. 빠르게 발전하는 ROS2의 모든 기능을 한 권에 담을 수는 없다. 그러나 구조와 원리를 이해하고 AI의 답을 검증하는 방법을 익히면, 새로운 센서나 로봇을 만나도 전체 흐름을 스스로 분석할 수 있다.

이 장을 마치면 다음 질문에 자신의 말로 답할 수 있다.

아직 명령어나 코드를 외울 필요는 없다. 먼저 큰 그림을 잡아 보자.

로봇은 하나의 프로그램이 아니다

작은 교육용 이동 로봇을 떠올려 보자. 겉으로는 바퀴가 달린 하나의 기계지만, 안에서는 서로 다른 일이 동시에 벌어진다.

- 카메라는 계속해서 영상을 만든다.
- 거리 센서는 주변 물체까지의 거리를 측정한다.

- 위치 추정 기능은 로봇이 어디에 있는지 계산한다.
- 판단 기능은 어느 방향으로 움직일지 결정한다.
- 제어 기능은 바퀴의 속도를 조절한다.
- 화면 표시 기능은 사람에게 현재 상태를 보여 준다.

이 기능을 하나의 거대한 프로그램에 모두 넣을 수도 있다. 처음에는 간단해 보이지만 기능이 늘어날수록 문제가 생긴다. 카메라 코드를 수정했는데 바퀴 제어가 멈추거나, 센서를 다른 제품으로 바꾸기 위해 전체 프로그램을 다시 고쳐야 할 수 있다. 한 부분에서 오류가 발생했을 때 어디서 잘못되었는지 찾기도 어렵다.

더 나은 방법은 기능을 작은 프로그램으로 나누는 것이다. 카메라 프로그램은 영상을 만들고, 인식 프로그램은 그 영상을 받아 물체를 찾는다. 이동 판단 프로그램은 인식 결과를 받아 방향을 결정하고, 모터 제어 프로그램은 그 명령을 실제 바퀴 움직임으로 바꾼다.

이제 중요한 문제가 남는다.

서로 따로 실행되는 프로그램들이 어떻게 상태를 발견하고, 약속된 형식으로 데이터를 주고받을까?

ROS2는 바로 이 문제를 해결하기 위한 공통 구조와 도구를 제공한다.



그림 1-2. 하나의 로봇도 감지, 판단, 제어, 상태 확인 같은 여러 기능의 협력으로 동작한다.

ROS2는 무엇인가

ROS는 **Robot Operating System**의 약자다. 이름에 Operating System이 들어 있지만 Windows나 Ubuntu처럼 컴퓨터 전체를 관리하는 일반적인 운영체제는 아니다. ROS2는 Ubuntu 같은 운영체제 위에서 실행되는 로봇 소프트웨어 개발용 프레임워크이자 도구 모음에 가깝다.

ROS2가 제공하는 대표적인 기능은 다음과 같다.

- 여러 프로그램이 서로를 발견하고 통신하는 방법
- 센서 데이터와 제어 명령에 사용할 표준 메시지 형식
- 프로그램을 묶어 배포하는 Package 구조
- 여러 프로그램을 한 번에 시작하는 Launch 기능
- 실행 중인 로봇 시스템을 관찰하는 명령행 도구
- 로봇의 좌표 관계를 관리하는 TF2
- 로봇 모양과 관절 구조를 표현하는 URDF
- 로봇 상태와 센서 데이터를 시각화하는 RViz2
- 시뮬레이터와 실제 로봇에 연결할 수 있는 생태계

ROS2 자체가 로봇을 대신 만들어 주지는 않는다. 카메라로 무엇을 인식할지, 장애물을 만났을 때 어떻게 행동할지, 모터를 얼마나 빠르게 돌릴지는 개발자가 설계해야 한다. 대신 ROS2는 기능을 나누고 연결하고 관찰하는 공통 기반을 제공한다.

ROS 1과 ROS2

인터넷이나 오래된 책을 검색하면 ROS 1 자료를 자주 만나게 된다. ROS2는 ROS 1의 단순한 업데이트가 아니라 통신 구조와 개발 방식이 크게 달라진 후속 세대다. ROS2는 분산 통신, 다양한 품질 정책, 여러 운영체제 지원, 실시간 시스템과 보안 등을 고려하여 새로 설계되었다.

이 책에서 ROS라고 짧게 부를 때도 특별한 설명이 없다면 ROS2를 의미한다. ROS 1 명령과 예제를 그대로 복사하면 동작하지 않을 수 있으므로 검색할 때는 ROS2 Jazzy처럼 세대와 배포판 이름을 함께 적는 습관을 들이는 것이 좋다.

작은 프로그램을 Node라고 부른다

ROS2에서는 하나의 책임을 맡아 실행되는 프로그램 단위를 **Node**라고 부른다. 반드시 하나의 파일이 Node 하나와 정확히 일치하는 것은 아니지만, 입문 단계에서는 “Node는 로봇의 한 가지 일을 맡은 실행 프로그램”이라고 이해하면 충분하다.

예를 들어 이동 로봇은 다음과 같이 Node별 역할과 데이터로 나눌 수 있다.

- **Camera Node**
 - 맡은 일: 카메라 영상을 읽는다.
 - 데이터: Image를 만들어 낸다.
- **Detector Node**
 - 맡은 일: 영상에서 목표물을 찾는다.
 - 데이터: 목표물 위치를 만들어 낸다.
- **Decision Node**
 - 맡은 일: 이동 방향을 결정한다.
 - 데이터: 목표 속도를 만들어 낸다.
- **Motor Node**
 - 맡은 일: 바퀴를 제어한다.
 - 데이터: Motor 명령을 사용한다.
- **Monitor Node**

- 말은 일: 상태를 화면에 표시한다.
- 데이터: Battery와 오류 상태를 사용한다.

Node를 작게 나누면 장점이 많다.

첫째, 한 기능만 집중해서 만들고 시험할 수 있다. 둘째, 카메라나 모터가 바뀌었을 때 관련 Node만 교체하기 쉽다. 셋째, 다른 사람이 만든 기능을 약속된 데이터 형식에 맞춰 재사용할 수 있다. 넷째, 문제가 생겼을 때 어느 Node까지 데이터가 정상적으로 전달되는지 단계별로 확인할 수 있다.

하지만 Node를 나누기만 해서는 협력할 수 없다. Node 사이에 데이터를 전달할 통신 방식이 필요하다.

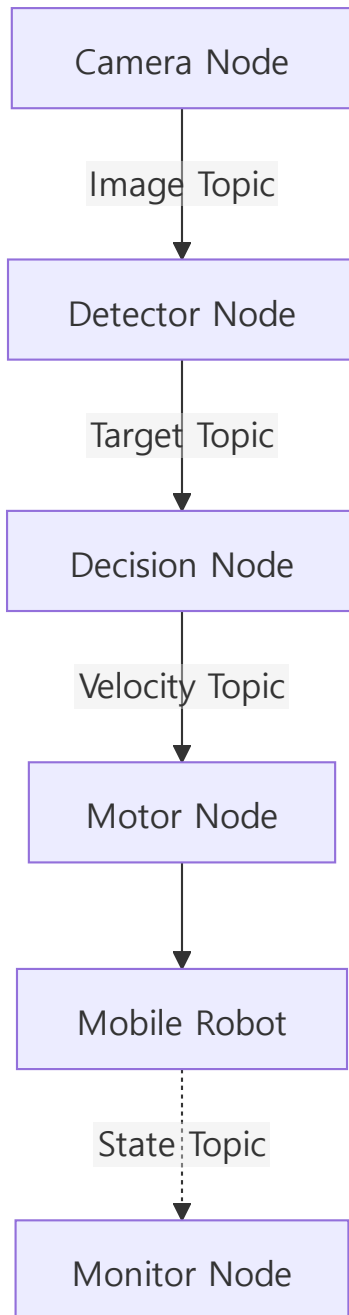


그림 1-3. Node는 각자 맡은 일을 수행하고 Topic으로 필요한 데이터를 전달한다.

Node가 대화하는 세 가지 기본 방법

ROS2 입문 과정에서 가장 먼저 만나는 통신 방식은 Topic, Service, Action이다. 각각 쓰임이 다르다.

Topic: 계속 흐르는 데이터

Topic은 센서 데이터나 속도 명령처럼 지속적으로 흐르는 정보를 전달할 때 사용한다. 데이터를 보내는 쪽을 Publisher, 받는 쪽을 Subscriber라고 한다.

카메라는 새 영상이 생길 때마다 발행하고, 영상이 필요한 여러 Node가 각자 구독할 수 있다. Publisher는 모든 Subscriber의 내부 동작을 기다리지 않는다. 이런 구조는 지속적인 데이터 흐름에 잘 맞는다.

Service: 질문 한 번, 응답 한 번

Service는 요청을 보내고 그 결과를 한 번 돌려받는 작업에 적합하다. “현재 설정값을 알려줘”, “새 거북이를 화면에 만들어 줘”처럼 비교적 짧게 끝나는 작업을 떠올리면 된다.

Action: 시간이 걸리는 목표

Action은 완료까지 시간이 걸리고 중간 진행 상황이나 취소가 필요한 작업에 사용한다. “목표 지점까지 이동하라”라는 Goal을 보내고, 이동 중에는 Feedback을 받으며, 끝나면 Result를 받는다. 필요하다면 도중에 Cancel을 요청할 수 있다.

지금 세 용어를 완벽히 구분하지 못해도 괜찮다. 이 책의 8장부터 19장까지 실제 코드를 작성하면서 차이를 반복해서 확인한다.

ROS2는 프로그램 사이의 약속을 만든다

Node끼리 통신하려면 데이터의 모양이 같아야 한다. 한쪽은 속도를 숫자 하나로 보내는데 다른 쪽은 앞뒤·좌우·회전 속도가 모두 들어올 것이라 기대한다면 통신할 수 없다.

ROS2에서는 Interface가 데이터의 구조를 정의한다. Topic에서 사용하는 Message, Service에서 사용하는 Service Interface, Action에서 사용하는 Action Interface가 대표적이다.

주소를 적을 때 “서울”이라고만 쓰는 것보다 국가, 도시, 도로명, 건물 번호를 정해진 칸에 적는 편이 정확하다. Interface도 이와 비슷하다. 보내는 쪽과 받는 쪽이 각 필드의 이름과 자료형을 같은 방식으로 이해하게 만든다.

이 약속 덕분에 Python으로 만든 Node와 C++로 만든 Node가 통신할 수 있고, 서로 다른 컴퓨터에서 실행되는 Node도 같은 ROS 네트워크에서 데이터를 교환할 수 있다. 이 책은 Python을 사용하지만 통신 상대가 반드시 Python일 필요는 없다.

무엇을 만들 수 있을까

ROS2는 특정 모양의 로봇만을 위한 도구가 아니다. 센서와 판단 기능, 움직임이 연결되는 다양한 시스템에 활용할 수 있다. 예를 들어 Wi-Fi로 연결된 온도 센서, 조명 제어기, 카메라의 데이터를 ROS2 Node가 주고받도록 구성하면 IoT 시스템에도 응용할 수 있다. 여러 컴퓨터에 작업을 나누는 분산 처리 구조도 만들 수 있으므로, 로봇을 넘어 다양한 장치와 시스템을 연결하는 데 활용할 수 있다.

이동 로봇

바퀴가 달린 로봇에 거리 센서나 카메라를 연결해 장애물을 피하고 목적지로 이동하게 만들 수 있다. 지도 작성, 위치 추정, 경로 계획 같은 기능을 단계적으로 결합할 수도 있다.

로봇 팔

관절 상태를 읽고 목표 자세를 전달하거나, 카메라로 물체를 찾아 집어 옮기는 시스템을 구성할 수 있다. 로봇 팔의 모양과 관절 관계를 모델로 만들고 시뮬레이션에서 먼저 시험할 수도 있다.

카메라와 AI 로봇

카메라 Node가 영상을 발행하고 AI Node가 사람이나 물체를 검출하도록 연결할 수 있다. 검출 결과는 화면 표시에만 사용할 수도 있고, 이동이나 로봇 팔 동작의 입력으로 사용할 수도 있다.

여러 로봇이 협력하는 시스템

Namespace를 사용하면 같은 종류의 Node와 Topic을 로봇별로 구분할 수 있다. 각 로봇의 상태를 모아 확인하거나 서로 다른 임무를 나눌 수 있다.

메이커 자동화 프로젝트

작은 이동 로봇, 카메라 팬-틸트 장치, 자동 급수 장치, 원격 감시 장치처럼 직접 만든 하드웨어에도 ROS2 구조를 적용할 수 있다. 처음부터 거대한 자율주행 시스템을 만들 필요는 없다. 센서 Node 하나와 출력 Node 하나를 연결하는 것부터 시작하면 된다.

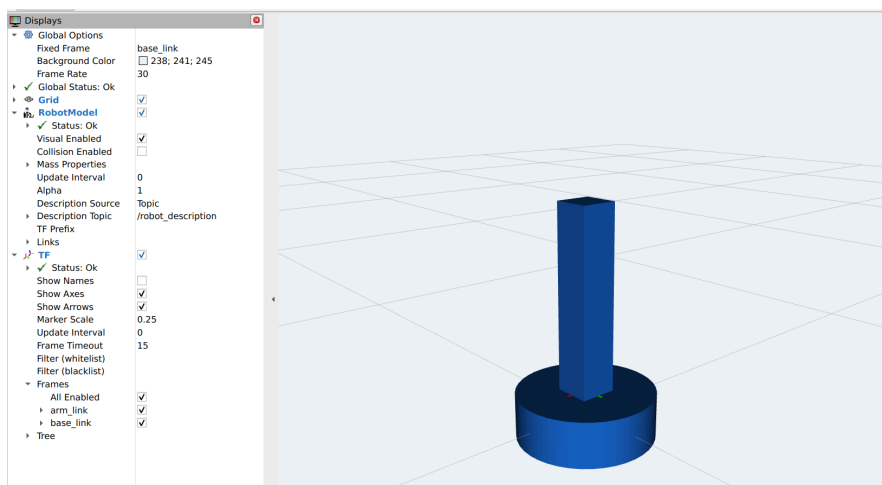


그림 1-4. URDF 모델과 TF를 RViz2에 표시한 실제 화면. 23~26장에서 Link와 Joint의 연결을 검증한다.

실제 로봇 없이도 시작할 수 있다

로봇을 배우려면 먼저 비싼 하드웨어를 사야 한다고 생각하기 쉽다. 그러나 소프트웨어 구조를 익히는 단계에서는 시뮬레이션이 오히려 편리한 경우가 많다.

시뮬레이션에서는 다음과 같은 장점이 있다.

- 잘못된 명령을 보내도 실제 장비가 파손되지 않는다.
- 센서와 로봇 모델을 여러 번 쉽게 초기화할 수 있다.
- 같은 조건을 반복하여 문제를 확인할 수 있다.

- 공간과 장비가 없어도 실습할 수 있다.
- 실행 화면과 통신 데이터를 함께 관찰하기 쉽다.

이 책에서는 세 가지 도구를 단계적으로 사용한다.

Turtlesim은 ROS2 통신을 배우기 위한 단순한 2차원 거북이 시뮬레이터다. 작은 도구지만 Node, Topic, Service, Action을 직접 관찰할 수 있다.

RViz2는 로봇이 이해하고 있는 정보를 시각화하는 도구다. 로봇 모델, 좌표계, 센서 데이터 등을 화면에서 확인한다. RViz2는 물리 시뮬레이터라기보다 ROS2 데이터 시각화 도구라는 점이 중요하다.

Gazebo Harmonic은 중력, 충돌, 센서와 로봇 움직임을 가상 환경에서 시험하는 시뮬레이터다. 이 책의 기준 환경인 ROS2 Jazzy와 권장 조합을 이룬다. 뒤의 장에서는 먼저 Turtlesim과 RViz2로 기본 원리를 익히고, 학습한 구조를 Gazebo 시뮬레이션으로 확장한다.

시뮬레이션에서 잘 작동했다고 해서 실제 로봇에서도 언제나 똑같이 움직이는 것은 아니다. 실제 센서에는 잡음이 있고, 바퀴는 미끄러지며, 통신이 지연될 수 있다. 그래서 시뮬레이션은 현실을 완전히 대신하는 도구가 아니라 안전하고 반복 가능한 첫 시험장으로 이해해야 한다.

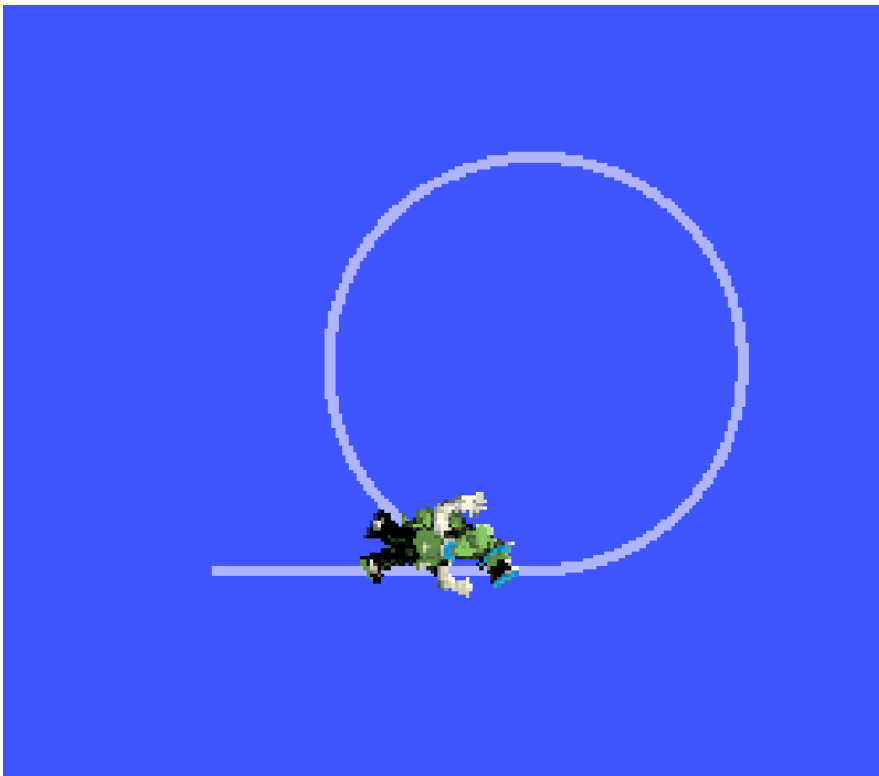


그림 1-5. Turtlesim에서 속도 명령으로 만든 곡선 궤적. 5장에서 직접 움직이며 Topic 통신을 확인한다.

이 책에서 사용할 기준 환경

ROS2와 Gazebo에는 여러 버전이 있다. 인터넷 예제의 버전이 서로 다르면 Package 이름이나 명령이 달라져 입문자가 혼란을 겪기 쉽다. 이 책은 다음 조합을 기준으로 설명한다.

구성 요소	기준
Ubuntu	24.04 LTS
ROS2	Jazzy Jalisco LTS
Gazebo	Harmonic LTS
주 언어	Python 3, rclpy
시각화	RViz2
첫 시뮬레이션	Turtlesim

Gazebo 공식 문서에서도 ROS2 Jazzy와 Gazebo Harmonic을 권장 조합으로 안내한다. 버전이 다른 환경에서도 ROS2의 기본 개념은 적용되지만, 이 책의 명령과 결과를 그대로 재현하려면 기준 조합을 사용하는 편이 좋다.

AI는 답안지가 아니라 함께 확인하는 조수다

ROS2를 공부하다 보면 긴 오류 메시지나 낯선 설정 파일을 자주 만나게 된다. 생성형 AI는 개념을 다른 방식으로 설명하거나, 오류 로그에서 확인할 항목을 정리하고, 코드 초안을 만드는 데 도움을 줄 수 있다.

하지만 AI가 제시한 Package나 함수가 실제로 존재하지 않을 수도 있다. 예전 ROS 1 방식과 ROS2 방식을 섞거나, Humble용 명령을 Jazzy에서도 같다고 가정할 수도 있다. 따라서 좋은 프롬프트는 질문을 자세히 쓰는 데서 끝나지 않는다. 실행 환경을 명시하고, 확인 방법까지 요구해야 한다.

다음 프롬프트를 AI 도구에 입력해 보자.

나는 ROS2를 처음 배우는 학생이다.

기준 환경은 Ubuntu 24.04, ROS2 Jazzy, Gazebo Harmonic, Python rclpy이다.

ROS2가 일반적인 Python 단일 프로그램과 다른 점을 작은 이동 로봇의 카메라, 판단, 바퀴 제어 예시로 설명해 줘.

다음 조건을 지켜 줘.

1. Node와 Topic이라는 용어를 포함한다.
2. ROS2가 일반 운영체제는 아니라는 점을 설명한다.
3. 입문자가 이해할 수 있는 비유를 하나 사용한다.
4. 확인되지 않은 내용은 추측이라고 표시한다.
5. 마지막에 내가 이해했는지 확인할 질문을 세 개 제시한다.

답변을 받은 뒤에는 다음을 확인한다.

- 이 책에서 설명한 Node와 Topic의 의미와 일치하는가?
- ROS 1 명령이나 다른 ROS2 배포판의 내용을 섞지 않았는가?
- 처음 보는 명령이나 API를 실제 존재 여부 확인 없이 믿고 있지 않은가?
- 설명을 자신의 말로 다시 표현할 수 있는가?

AI에게 질문하는 능력과 AI의 답을 검증하는 능력은 함께 길러야 한다. 이 책에서는 각 장의 “AI와 함께 문제 해결하기” 부분에서 이 과정을 반복한다.

앞으로의 학습 여행

이 책은 한 번에 복잡한 로봇을 만드는 방식으로 진행하지 않는다. 작은 성공을 쌓으며 전체 구조를 넓혀 간다.

1. Terminal과 개발 환경을 준비한다.
2. Turtlesim을 실행하고 ROS 그래프를 관찰한다.
3. Python으로 자신의 첫 Node를 만든다.
4. Publisher와 Subscriber를 연결한다.
5. Service, Action, Parameter를 사용한다.
6. 여러 로봇과 좌표계를 구분한다.
7. URDF와 Xacro로 로봇 모델을 만든다.
8. RViz2와 Gazebo에서 결과를 확인한다.



그림 1-6. 작은 시뮬레이션에서 시작하여 기능을 하나씩 연결하면 자신의 로봇 프로젝트로 확장할 수 있다.

중요한 것은 코드를 많이 입력하는 것이 아니라 “어떤 Node가 어떤 데이터를 만들고, 누가 그 데이터를 사용하며, 결과를 어떻게 확인할 것인가”를 계속 질문하는 습관이다. 코드를 직

접 입력하는 연습도 중요하지만, AI 시대에서는 구조를 파악하고 질문을 할 수 있어야 한다. 이 관점이 잡히면 새로운 센서나 로봇을 만나도 전체 흐름을 스스로 분석할 수 있다.

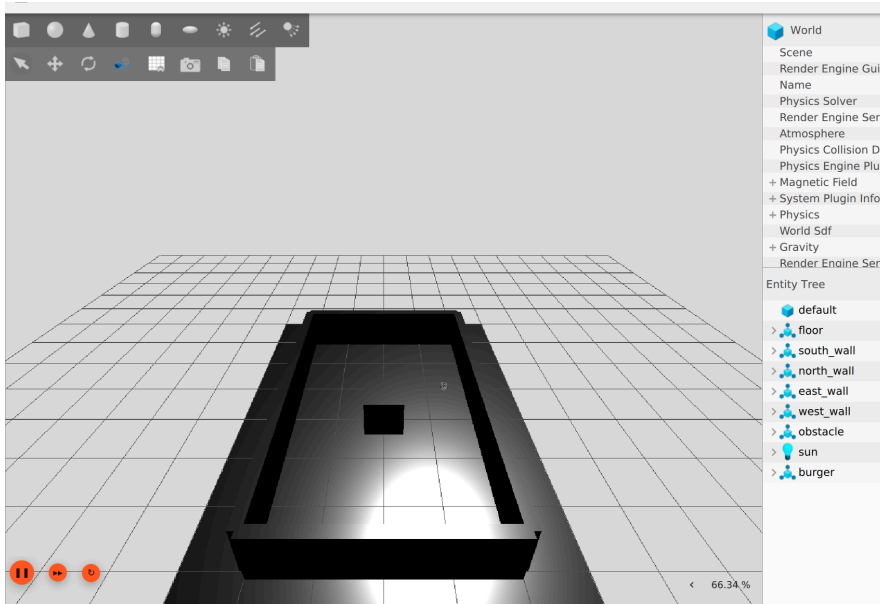


그림 1-7. Gazebo Sim에서 실행한 장애물 공간. 27장에서 LaserScan과 속도 명령을 연결한다.

1장 마무리

핵심 정리

- 로봇은 센서, 판단, 제어, 시각화 등 여러 기능이 협력하는 시스템이다.
- ROS2는 일반 운영체제가 아니라 로봇 소프트웨어를 나누고 연결하기 위한 프레임워크와 도구 모음이다.
- ROS2에서 한 가지 책임을 맡은 실행 단위를 Node라고 부른다.
- Node는 Topic, Service, Action 같은 방식을 목적에 맞게 사용하여 통신한다.
- Interface는 프로그램 사이에서 주고받을 데이터의 구조를 정의한다.
- Turtlesim, RViz2, Gazebo를 사용하면 실제 로봇 없이도 ROS2의 핵심 구조를 배울 수 있다.
- AI의 답은 실행 환경, 공식 문서, 실제 빌드와 실행 결과로 검증해야 한다.

스스로 확인하기

1. ROS2의 이름에 Operating System이 들어 있지만 Ubuntu와 같은 운영체제가 아닌 이유를 설명해 보자.
2. 카메라 영상 처리와 바퀴 제어를 서로 다른 Node로 나누면 어떤 장점이 있는가?
3. 카메라 영상처럼 계속 흐르는 데이터에는 Topic, Service, Action 중 무엇이 가장 적합한가?
4. 시간이 오래 걸리고 중간 진행 상황과 취소가 필요한 작업에는 어떤 통신 방식이 적합한가?
5. 시뮬레이션 실습의 장점과 한계를 각각 한 가지씩 말해 보자.

실습 완료 체크리스트

- ☐ ROS2가 해결하려는 문제를 한 문장으로 설명할 수 있다.
- ☐ Node를 로봇 기능의 예와 연결해 설명할 수 있다.
- ☐ Topic, Service, Action의 대표 용도를 구분할 수 있다.
- ☐ Turtlesim, RViz2, Gazebo의 역할이 서로 다를 수 있다.
- ☐ 이 책의 기준 환경을 확인할 수 있다.

- AI 질문에 환경과 검증 조건을 포함해야 하는 이유를 설명할 수 있다.

참고 자료

- ROS2 Jazzy 공식 문서: 기본 개념
<https://docs.ros.org/en/jazzy/Concepts/Basic.html>
- ROS2 Jazzy 공식 문서: Topic·Service·Action
<https://docs.ros.org/en/jazzy/Concepts/Basic/Interfaces-Topics-Services-Actions.html>
- ROS2 Jazzy 공식 문서: Turtlesim·ros2·rqt 시작하기
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Introducing-Turtlesim/Introducing-Turtlesim.html>
- Gazebo Harmonic 공식 문서: ROS와 Gazebo 버전 조합
https://gazebosim.org/docs/harmonic/ros_installation/

다음 장에서는 Windows, WSL2, Ubuntu가 어떻게 연결되는지 살펴보고 ROS2를 실행할 장소를 정확히 구분한다.



Windows·WSL2·Ubuntu의 관계

02 CHAPTER

이 책은 Windows 11의 WSL2에 설치한 Ubuntu에서 ROS2를 실습하는 환경을 기준으로 한다.
ROS2는 Ubuntu 안에서 실행되며, WSL2는 Windows에서 이 Linux 환경을 사용할 수 있게 한다.
이 장에서는 각 환경의 역할과 명령을 입력할 위치, 프로젝트 파일을 저장할 위치를 구분한다.

학습 목표

- Windows, WSL2, Ubuntu, ROS2는 각각 어떤 역할을 하는가?
- WSL2는 단순한 Terminal 프로그램인가, 가상 머신인가?
- PowerShell과 Ubuntu Terminal에는 각각 어떤 명령을 입력해야 하는가?
- ROS2 프로젝트를 /mnt/c보다 /home 아래에 두는 이유는 무엇인가?
- Windows에서 보이는 VS Code와 Ubuntu에서 실행되는 개발 도구는 어떻게 연결되는가?
- Turtlesim이나 RViz2 창이 Windows 바탕 화면에 나타나도 Linux 프로그램이라고 말할 수 있는 이유는 무엇인가?

이 장의 학습 지도

01 Windows

02 WSL2 · Ubuntu

03 ROS2 개발 환경

완료 기준

01 PowerShell의 `wsl -l -v`와 Ubuntu의 `/etc/os-release`에서 배포판·WSL 버전·Ubuntu 버전을 찾아 기록한다.

02 Windows → WSL2 → Ubuntu → ROS2의 관계를 그리고, 명령을 입력할 Terminal을 구분한다.

03 /home/mnt/c-Windows 경로의 관계를 설명하고 프로젝트를 둘 Linux 홈 경로를 적는다.

04 VS Code의 WSL 연결 표시와 Terminal의 `pwd`를 비교해 편집 파일과 실행 도구가 같은 Ubuntu에 있음을 확인한다.

개념을 이해하고 · 직접 실행하고 · 결과로 확인하기

컴퓨터에 Ubuntu를 직접 설치해서 실습할 수도 있다. 이 책에서는 Windows를 사용하는 독자가 기존 환경을 유지하며 따라올 수 있도록 WSL2를 사용한다. Ubuntu를 직접 설치한 경우에도 ROS2의 기본 개념과 Ubuntu Terminal에서 실행하는 명령은 함께 학습할 수 있다.

Windows에서 ROS2를 처음 시작하면 뜻밖의 질문을 만나게 된다. 분명 Windows 노트북을 샀는데 Ubuntu를 설치하라고 하고, PowerShell과 Ubuntu Terminal을 번갈아 사용하며, VS Code 화면 아래에는 WSL이라는 표시가 나타난다. 파일 하나도 C:\Users와 /home 중 어디에 저장해야 할지 헷갈린다.

이 혼란은 각 도구의 역할을 한꺼번에 이해하려 하기 때문에 생긴다. 먼저 한 문장으로 정리해 보자.

Windows는 컴퓨터의 주 운영체제이고, WSL2는 Windows 안에서 Linux 환경을 실행하는 기반이며, Ubuntu는 그 기반에서 사용하는 Linux 배포판이다. ROS2는 Ubuntu 위에 설치하여 실행한다.

이 장에서는 ROS2를 아직 설치하지 않는다. 대신 앞으로 모든 실습의 바탕이 되는 환경의 구조를 이해하고, 명령을 어느 Terminal에 입력할지, 프로젝트 파일을 어디에 저장할지 판단하는 연습을 한다.



그림 2-1. Windows를 계속 사용하면서 WSL2의 Ubuntu에서 ROS2 도구와 로봇 시뮬레이션을 실행할 수 있다.

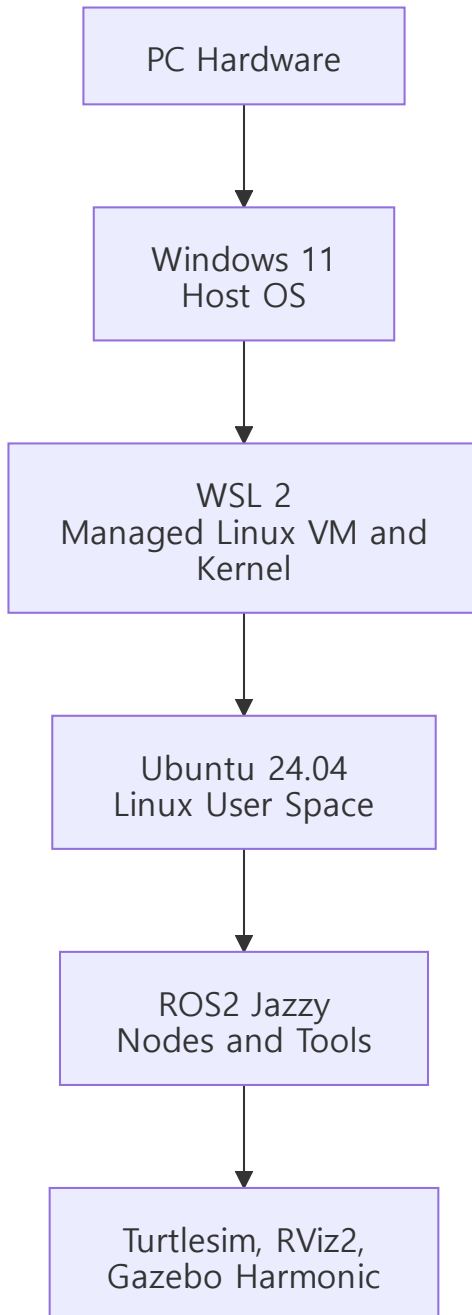


그림 2-2. 이 책의 실습 환경은 Windows 위의 WSL2, Ubuntu, ROS2가 차례로 쌓인 구조다.

이 장을 마치면 다음 질문에 답할 수 있다.

먼저 네 가지 이름을 분리해 보자

Windows: 컴퓨터 전체를 관리하는 주 운영체제

이 책에서 Windows는 노트북을 켜올 때 가장 먼저 실행되는 **Host Operating System**, 즉 주 운영체제다. 화면, 키보드, 저장 장치, 네트워크 같은 하드웨어를 관리하고 Windows용 프로그램을 실행한다. 웹 브라우저와 Windows용 VS Code도 여기에 속한다.

Windows의 대표적인 명령행 환경은 PowerShell이다. 다음과 같은 Prompt가 보이면 현재 PowerShell을 사용하고 있을 가능성이 높다.

```
PS C:\Users\gildong>
```

C:\Users\gildong은 Windows 방식의 경로다. 드라이브 문자인 C:, 역슬래시 \, 사용자 폴더 이름이 보인다.

WSL: Windows와 Linux를 이어 주는 기능

WSL은 **Windows Subsystem for Linux**의 약자다. Windows에서 Linux 배포판과 Linux 명령행 도구를 사용할 수 있게 해 주는 Windows 기능이다. 우리가 직접 다루는 것은 현재 기본 방식인 WSL2다.

WSL2는 Linux 명령을 Windows 명령으로 흉내 내는 Terminal 꾸미기 프로그램이 아니다. 실제 Linux Kernel을 가벼운 가상 머신 안에서 실행한다. 다만 일반적인 가상 머신 프로그램처럼 사용자가 CPU 수, 가상 디스크, 화면 장치 등을 매번 직접 관리하지 않아도 되도록 Windows와 긴밀하게 통합되어 있다.

따라서 다음 두 설명을 함께 기억해야 한다.

- WSL2 안의 Ubuntu는 실제 Linux Kernel 위에서 동작한다.
- 그 Linux 환경의 시작, 종료, 파일 접근, 화면 표시는 Windows와 편리하게 연결된다.

WSL은 Ubuntu 그 자체가 아니다. WSL은 Linux 환경을 실행하는 기반이고, 그 위에 Ubuntu 나 다른 Linux 배포판을 설치할 수 있다. 한 컴퓨터에 여러 배포판을 함께 설치하는 것도 가능하다.

Ubuntu: WSL에서 사용할 Linux 배포판

Linux라는 말은 좁게는 Kernel을 뜻한다. 실제 사용자는 Kernel만으로 개발하지 않는다. 명령어, Package 관리자, System Library, 기본 프로그램이 함께 묶인 배포판을 사용한다. Ubuntu는 널리 쓰이는 Linux 배포판 중 하나다.

이 책의 기준은 **Ubuntu 24.04 LTS**다. Ubuntu Terminal을 열면 대개 다음과 비슷한 Prompt가 나타난다.

```
gildong@my-pc:~$
```

여기서 gildong은 Ubuntu 사용자 이름, my-pc는 Host 이름, ~는 현재 사용자의 Home Directory를 뜻한다. Windows 사용자 이름과 Ubuntu 사용자 이름은 같게 정할 수 있지만

서로 별개의 계정이다.

ROS2: Ubuntu 위에서 실행하는 개발 환경

ROS2는 운영체제가 아니다. Ubuntu 위에 설치되는 Library, 통신 기반, 명령행 도구, 개발 규칙의 모음이다. ros2 명령, Python의 rclpy, RViz2와 Turtlesim 같은 프로그램은 Ubuntu 환경 안에서 실행된다.

이 책에서 사용할 전체 관계는 다음과 같다.

1. 물리적인 노트북이 있다.
2. Windows 11이 노트북을 관리한다.
3. WSL2가 실제 Linux Kernel을 사용하는 관리형 가상 환경을 제공한다.
4. Ubuntu 24.04가 Linux 명령과 Package 환경을 제공한다.
5. ROS2 Jazzy가 Ubuntu 위에서 Node와 개발 도구를 실행한다.
6. Turtlesim, RViz2, Gazebo Harmonic으로 결과를 관찰한다.

문제가 생겼을 때도 이 순서를 따라 확인하면 좋다. ROS2 명령이 없다고 바로 ROS2를 재설치하기보다, 지금 연 Terminal이 Ubuntu인지, Ubuntu 배포판이 24.04인지, 환경 설정을 불러왔는지부터 확인할 수 있다.

WSL2는 무엇과 비슷하고 무엇이 다른가

Dual Boot와 다르다

Dual Boot는 컴퓨터를 켤 때 Windows와 Ubuntu 중 하나를 선택해 실행한다. Ubuntu를 사용하는 동안에는 Windows 프로그램을 함께 실행하기 어렵고, 운영체제를 바꾸려면 보통 재부팅해야 한다.

WSL2에서는 Windows를 켜 상태로 Ubuntu Terminal과 Linux 프로그램을 실행한다. 브라우저에서 문서를 보고, Windows용 VS Code로 코드를 편집하고, Ubuntu에서 ROS2를 실행할 수 있다. 입문 실습에서 WSL2가 편리한 이유다.

전통적인 가상 머신과 비슷하지만 관리 방식이 다르다

WSL2도 가상화 기술과 실제 Linux Kernel을 사용한다는 점에서는 가상 머신과 닮았다. 그러나 사용자가 별도의 가상 화면을 열고 Ubuntu Desktop 전체를 조작하는 방식이 아니다. Windows가 가상 머신의 수명과 자원을 관리하고, Terminal·파일·Network·GUI 연결을 통합한다.

그러므로 WSL2 Ubuntu에 완전한 Ubuntu Desktop 화면이 보이지 않는 것은 문제가 아니다. ROS2 실습에는 Ubuntu Terminal과 필요한 GUI Application만 있으면 된다.

Container와도 다르다

Container는 보통 Application과 의존성을 격리해 반복 배포하기 위한 기술이다. WSL2는 Windows에서 Linux 개발 환경을 제공하는 더 아래쪽 기반이다. 나중에 WSL2 안에서 Docker Container를 실행할 수도 있지만, 이 책의 기본 실습에는 Container가 필요하지 않다.

입문 단계에서는 “Windows 안에 ROS2 개발용 Ubuntu 컴퓨터가 하나 더 있다”고 생각하면 이해하기 쉽다. 다만 그 컴퓨터가 Windows와 긴밀하게 연결되어 있다는 점을 덧붙이면 더 정확하다.

ROS2 MAKER GUIDE / CHAPTER 02 / 01

```
$ uname -sr
```

```
Linux 6.18.33.2-microsoft-standard-WSL2
```

그림 2-3. WSL2에서 조회한 Linux Kernel. 명령이 실행되는 운영체제를 확인한다.

지금 어느 Terminal에 있는가

가장 흔한 오류는 명령 자체가 아니라 **명령을 입력한 장소**가 틀려서 발생한다. 같은 검은색 창처럼 보여도 PowerShell과 Ubuntu Terminal은 서로 다른 명령 환경이다.

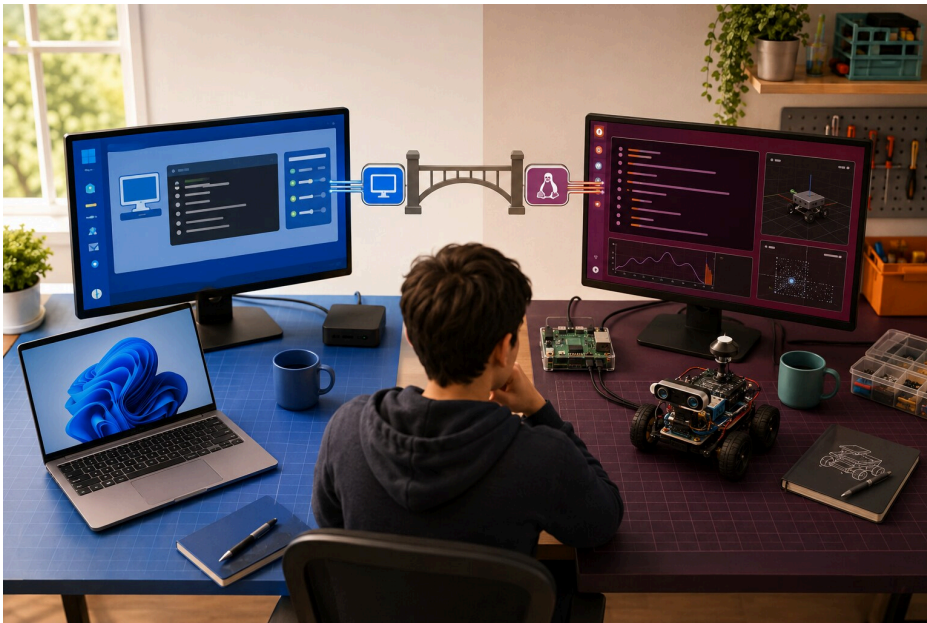


그림 2-4. PowerShell은 Windows와 WSL을 관리하고, Ubuntu Terminal은 Linux Package와 ROS2 개발 도구를 실행한다.

PowerShell에서 할 일

PowerShell은 WSL 자체를 설치하고 상태를 관리할 때 사용한다.

```
wsl --status
wsl --list --verbose
```

```
wsl --version
```

`wsl --list --verbose`는 짧게 `wsl -l -v`라고 입력할 수 있다. 다음과 비슷한 결과가 나오면 Ubuntu-24.04가 WSL2 방식으로 등록된 것이다.

NAME	STATE	VERSION
* Ubuntu-24.04	Running	2

여기서 확인할 핵심은 배포판 이름과 **VERSION** 값이다. **STATE**는 사용 중이면 **Running**, 완전히 종료되어 있으면 **Stopped**가 될 수 있으므로 어느 쪽이든 반드시 오류는 아니다.

WSL 전체를 안전하게 종료한 뒤 다시 시작하고 싶다면 모든 Ubuntu Terminal과 WSL 연결 VS Code 창을 닫고 PowerShell에서 다음 명령을 사용할 수 있다.

```
wsl --shutdown
```

이 명령은 실행 중인 모든 WSL 배포판을 종료한다. 작업 중인 프로그램이 있다면 먼저 저장하고 종료해야 한다.

Ubuntu Terminal에서 할 일

Ubuntu Package 설치, Git 작업, Python 실행, ROS2와 `colcon` 명령은 Ubuntu Terminal에서 수행한다.

```
pwd
whoami
cat /etc/os-release
uname -r
printenv WSL_DISTRO_NAME
```

각 명령이 알려 주는 내용은 다음과 같다.

- `pwd`: 현재 Directory의 Linux 경로
- `whoami`: 현재 Ubuntu 사용자 이름
- `cat /etc/os-release`: Ubuntu 배포판과 Version 정보
- `uname -r`: 현재 Linux Kernel 정보
- `printenv WSL_DISTRO_NAME`: 현재 WSL 배포판 이름

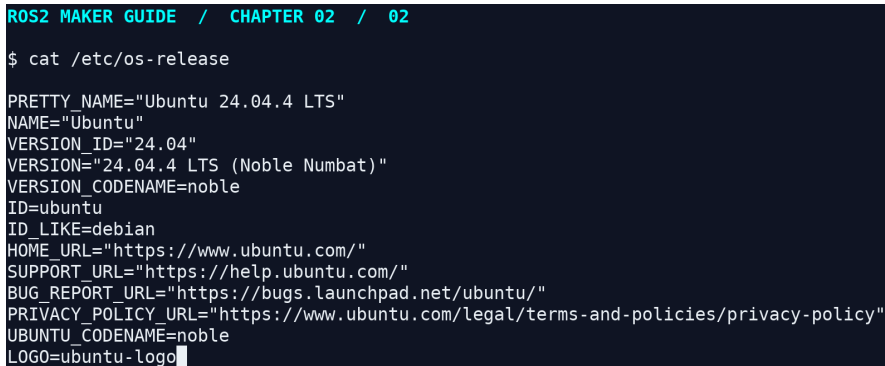
`WSL_DISTRO_NAME`이 비어 있다고 해서 무조건 고장은 아니다. Native Ubuntu나 일반 가상 머신에서는 이 WSL 전용 환경 변수가 존재하지 않는다. 여러 단서를 함께 보고 현재 환경을 판단해야 한다.

명령을 입력할 위치 빠르게 찾기

- WSL을 설치하거나 배포판 목록을 본다
 - 위치: Windows PowerShell

- 예: `wsl --install, wsl -l -v`
- **WSL을 완전히 다시 시작한다**
 - 위치: Windows PowerShell
 - 예: `wsl --shutdown`
- **Ubuntu Package를 설치한다**
 - 위치: Ubuntu Terminal
 - 예: `sudo apt update`
- **ROS2 Node를 실행하거나 확인한다**
 - 위치: Ubuntu Terminal
 - 예: `ros2 node list`
- **Workspace를 Build한다**
 - 위치: Ubuntu Terminal
 - 예: `colcon build`
- **Windows 파일을 탐색한다**
 - 위치: Windows Explorer
- **ROS2 소스 코드를 편집한다**
 - 위치: WSL로 연결된 VS Code

PowerShell에서 `sudo apt update`를 입력하거나 Ubuntu Terminal에서 `wsl --status`를 입력하면 명령을 찾을 수 없다는 메시지가 나올 수 있다. 오류 문구만 검색하기 전에 Prompt와 현재 Directory를 먼저 확인하자.



```

ROS2 MAKER GUIDE / CHAPTER 02 / 02

$ cat /etc/os-release

PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"
UBUNTU_CODENAME=noble
LOGO=ubuntu-logo

```

그림 2-5. Ubuntu 배포판 정보 조회 결과. Windows 버전과 Ubuntu 버전을 구분한다.

하나의 디스크를 바라보는 서로 다른 경로

Windows와 Ubuntu는 경로를 표현하는 방식이 다르다.

```

Windows: C:\Users\gildong\Documents
Ubuntu: /home/gildong

```

Windows는 C: 같은 Drive Letter와 역슬래시를 사용한다. Linux는 /에서 시작하는 하나의 Directory Tree와 슬래시를 사용한다.

Ubuntu에서 Windows 파일 보기

WSL은 Windows의 C: Drive를 일반적으로 /mnt/c 아래에 연결한다.

```
cd /mnt/c/Users  
ls
```

따라서 Windows의 다음 파일은 Ubuntu에서 서로 같은 파일을 가리킨다.

```
C:\Users\gildong\Documents\memo.txt  
/mnt/c/Users/gildong/Documents/memo.txt
```

이 기능은 Windows에 저장된 문서를 잠깐 읽거나 결과 파일을 옮길 때 편리하다. 그러나 “접근할 수 있다”와 “ROS2 프로젝트의 작업 장소로 가장 좋다”는 같은 말이 아니다.

Windows에서 Ubuntu 파일 보기

Ubuntu Terminal의 현재 Directory를 Windows Explorer로 열려면 다음 명령을 사용한다.

```
explorer.exe .
```

마지막의 점 .은 현재 Directory를 뜻한다. Windows Explorer 주소창에서 \\wsl\$을 입력해 설치된 배포판의 Linux 파일을 볼 수도 있다. 파일을 확인하거나 복사할 때 유용하지만, Linux 프로젝트 내부 파일을 Windows 전용 편집기로 무분별하게 수정하면 권한이나 줄바꿈 문제가 생길 수 있다. WSL에 연결된 VS Code를 사용하는 편이 안전하다.

ROS2 프로젝트는 /home 아래에 둔다

Microsoft도 Linux 명령행 도구로 작업하는 프로젝트는 WSL의 Linux File System에 저장할 것을 권장한다. 이 책에서는 다음과 같은 위치를 사용한다.

```
/home/gildong/ros2_ws
```

Home Directory 기호 ~를 사용하면 사용자 이름과 관계없이 다음처럼 표현할 수 있다.

```
~/ros2_ws
```

권장하지 않는 작업 위치는 다음과 같다.

```
/mnt/c/Users/gildong/ros2_ws
```

왜 구분해야 할까?

첫째, 많은 작은 파일을 읽고 쓰는 Git, Python, colcon Build는 Linux File System에서 더 빠른 경우가 많다. /mnt/c를 통한 Cross-OS File Access에는 추가 비용이 생긴다.

둘째, Linux의 실행 권한, Symbolic Link, 파일 소유권 같은 성질을 더 자연스럽게 사용할 수 있다. ROS2 Package와 Build 도구는 이러한 Linux 동작을 전제로 하는 경우가 많다.

셋째, Windows는 보통 파일 이름의 대소문자를 구분하지 않지만 Linux는 구분한다.

Robot.py와 robot.py가 Linux에서는 다른 파일이 될 수 있다. 두 File System을 오가며 개발하면 이런 차이가 예상치 못한 오류를 만들 수 있다.

정리하면, **Linux 도구로 작업하는 파일은 Linux File System에 둔다**. 완성된 EPUB처럼 Windows Application으로 열어야 하는 결과물은 필요할 때 Explorer로 접근하거나 복사하면 된다.

VS Code는 어느 쪽에서 실행되는가

Windows용 VS Code 화면을 보고 있으면서 Ubuntu의 소스 코드를 편집할 수 있다. 이것은 VS Code의 Remote - WSL 연결이 화면과 개발 도구의 실행 위치를 나누기 때문이다.

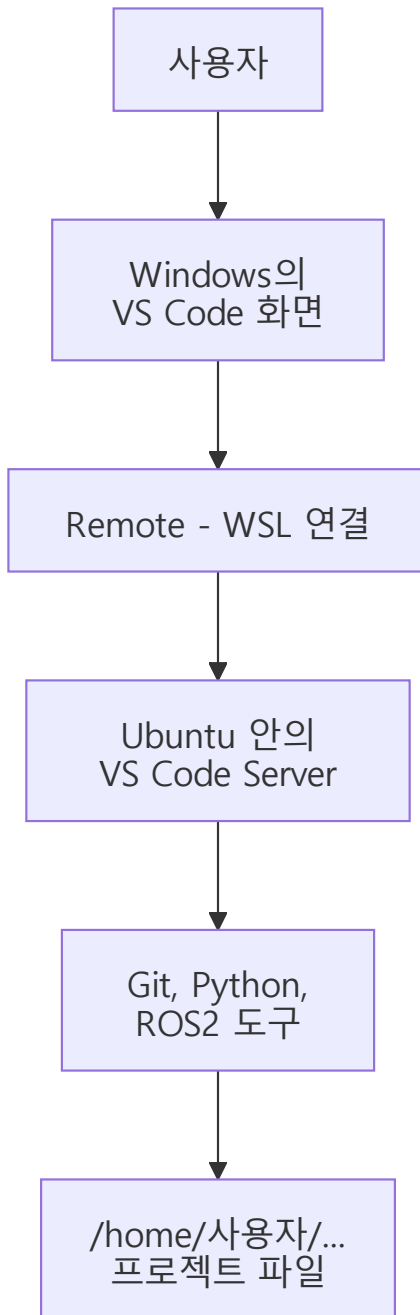


그림 2-6. 화면은 Windows에 있지만 Git, Python, ROS2와 프로젝트 파일은 Ubuntu 안에서 사용한다.

Ubuntu Terminal에서 프로젝트 Directory로 이동한 뒤 다음 명령을 실행한다.

```
cd ~/ros2_ws  
code .
```

VS Code 창이 열리면 다음을 확인한다.

1. 창 왼쪽 아래에 WSL: Ubuntu-24.04와 비슷한 표시가 있는가?
2. VS Code에서 새 Terminal을 열었을 때 Linux Prompt가 나타나는가?
3. Terminal에서 pwd를 실행했을 때 /home/... 경로가 나오는가?
4. Python, Git, ROS 관련 Extension이 WSL 환경에 설치되어 있는가?

Remote - WSL 방식에서는 Windows의 VS Code가 화면과 Keyboard 입력을 담당하고, Ubuntu 안의 VS Code Server가 파일 접근, Terminal, Extension, Git, Language Tool을 담당한다. 그래서 Windows용 VS Code를 다시 설치할 필요도 없고 Ubuntu Desktop 전체를 띄울 필요도 없다.

단순히 Windows VS Code에서 \\wsl\$ 경로의 파일을 연 것과 Remote - WSL로 연결한 것은 다르다. 파일만 보인다고 개발 도구가 Ubuntu에서 실행되는 것은 아니다. 왼쪽 아래 WSL 표시와 Terminal의 pwd를 함께 확인해야 한다.

Linux 프로그램의 창이 Windows에 나타나는 이유

Turtlesim, RViz2, RQt, Gazebo는 화면이 필요한 Linux GUI Application이다. WSL2는 WSLg라는 통합 기능을 통해 Linux GUI Application의 창을 Windows Desktop에 표시할 수 있다. 창의 테두리와 Taskbar는 Windows처럼 보여도 실제 Process는 Ubuntu 안에서 실행될 수 있다.

예를 들어 Ubuntu Terminal에서 Turtlesim을 실행하면 거북이 창은 Windows Desktop에 나타난다. 이때 다음 관계가 성립한다.

- 실행 명령과 ROS2 환경: Ubuntu
- Turtlesim Process: Ubuntu의 Linux Process
- 창 표시와 입력: WSLg를 거쳐 Windows Desktop에 통합

그러므로 “창이 Windows에 보인다”는 사실만으로 Windows용 프로그램이라고 판단하면 안 된다. 어느 Terminal에서 실행했는지와 어느 환경에 Package가 설치되었는지를 확인해야 한다.

GUI 창이 열리지 않을 때는 먼저 실행 중인 Ubuntu와 Windows가 WSLg를 지원하는 상태인지 확인한다. PowerShell에서 WSL을 Update하고 종료한 뒤 다시 실행하는 방법도 있다.

```
wsl --update
wsl --shutdown
```

이 명령을 사용하기 전에는 WSL 안의 작업을 저장하고 Process를 종료한다. 다음 장에서 실제 설치와 화면 실행을 단계별로 확인한다.

ROS2 MAKER GUIDE / CHAPTER 02 / 03

```
$ pwd; printf "DISPLAY=%s\nWAYLAND_DISPLAY=%s\n" "$DISPLAY" "$WAYLAND_DISPLAY"; df -h ~
/home/aa/gong_manipulator_20206
DISPLAY=:0
WAYLAND_DISPLAY=wayland-0
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdd        1007G  26G  931G   3% /
```

그림 2-7. Linux 홈 경로와 DISPLAY·WAYLAND_DISPLAY 확인 결과. GUI 연결 환경과 파일 위치를 함께 확인한다.

WSL2의 Network를 이해하는 첫걸음

ROS2는 Network를 통해 Node를 발견하고 데이터를 주고받는다. 한 컴퓨터 안에서 Turtlesim을 실행하는 동안에는 Network 구조를 깊이 알지 않아도 된다. 그래도 WSL2가 Windows와 완전히 같은 Network 공간은 아니라는 사실은 알아둘 필요가 있다.

기본 설정의 WSL2는 가상 Network Adapter와 NAT 방식으로 Windows Network에 연결된다. Ubuntu는 자체 IP Address를 가질 수 있고 WSL을 다시 시작하면 Address가 달라질 수 있다. Windows와 WSL 사이의 localhost 접근은 편리하게 통합되어 있지만, 외부 컴퓨터나 실제 로봇에서 WSL의 ROS2 Node를 발견하려면 방화벽, Network Mode, DDS 설정을 추가로 확인하고 설정을 수정할 필요가 있다. DDS 통신은 UDP Multicast를 사용하므로 단순히 Port를 열어 주는 것만으로는 충분하지 않다. 같은 subnet에 있어야 하고, DDS 설정이 호환되어야 한다. 그러기 위해서 Network Adapter 방식을 mirror로 바꾸고 Windows 방화벽의 inbound와 outbound rule을 추가해야 한다. 이 책에서는 WSL2의 Network 구조를 깊이 다루지 않는다. 나중에 실제 로봇과 연결할 때 필요한 최소한의 설정만 안내한다.

이 책의 초기 실습은 한 PC의 WSL2 안에서 모든 ROS2 Node를 실행한다. 따라서 IP Address를 고정하거나 Port를 직접 열 필요가 없다. 나중에 실제 로봇이나 다른 PC를 연결할 때 다음 항목을 구분해서 다룬다.

- Windows Firewall이 통신을 허용하는가?
- WSL2의 Network Mode가 무엇인가?
- 양쪽 장치가 같은 Network에 있는가?
- ROS2 Domain ID와 DDS 설정이 호환되는가?

지금은 Network 문제가 생겼을 때 “ROS2만의 문제”라고 단정하지 말고 Windows, WSL2, Ubuntu, DDS 계층을 나누어 확인한다는 원칙만 기억하자.

따라 하기: 내 실행 환경 지도 만들기

이 실습은 프로그램을 설치하거나 설정을 바꾸지 않는다. 현재 환경을 관찰하고 결과를 기록한다.

1단계: PowerShell에서 WSL 확인하기

Windows Terminal이나 PowerShell을 열고 다음 명령을 실행한다.

```
wsl --status
wsl -l -v
```

```
wsl --version
```

다음 질문에 답을 적는다.

- 기본 배포판의 이름은 무엇인가?
- Ubuntu 배포판의 WSL Version은 2인가?
- WSL과 Kernel Version은 무엇인가?

명령이 인식되지 않는다면 WSL이 아직 설치되지 않았거나 Windows Version이 요구 조건을 충족하지 않을 수 있다. 결과를 보관한 뒤 이 장의 참고 자료에 있는 Microsoft WSL 설치 안내로 환경을 준비한다. Ubuntu가 실행되는 것을 확인한 다음 3장의 ROS2 설치로 진행한다.

2단계: Ubuntu의 정체 확인하기

Ubuntu Terminal을 열고 다음 명령을 실행한다.

```
whoami
pwd
cat /etc/os-release
uname -r
```

/etc/os-release 결과에서 VERSION_ID="24.04"를 찾는다. pwd 결과가 /home/사용자이름이라면 Linux Home Directory에 있는 것이다.

3단계: 양쪽 File System 연결 확인하기

Ubuntu Terminal에서 다음 명령을 실행한다.

```
cd ~
explorer.exe .
```

Windows Explorer가 열리고 Ubuntu Home Directory의 파일이 보이는지 확인한다. 이 기능은 파일을 찾는 통로이지, 프로젝트를 Windows File System으로 옮기라는 뜻은 아니다.

이제 Windows의 C: Drive가 Ubuntu에서 보이는지 확인한다.

```
ls /mnt/c
```

Program Files, Users, Windows 같은 Directory가 보일 수 있다. 환경과 언어 설정에 따라 목록은 다를 수 있다.

4단계: 작업 위치 판정하기

앞으로 사용할 저장소 Directory에서 다음 명령을 실행한다.

```
pwd
git rev-parse --show-toplevel
```

두 번째 명령은 현재 위치가 Git 저장소 안일 때 최상위 Directory를 보여 준다. 결과가 `/home/...`으로 시작하면 권장 위치다. `/mnt/c/...`로 시작한다면 중요한 작업을 시작하기 전에 4장의 Git 절차를 따라 Linux Home Directory에 Clone하는 편이 좋다.

아직 저장소가 없어서 `not a git repository` 오류가 나오는 것은 정상이다. 이 단계의 목적은 오류를 없애는 것이 아니라 명령이 무엇을 확인하는지 이해하는 것이다.

5단계: 환경 기록 남기기

다음 양식을 자신의 학습 기록에 복사하여 채운다.

```
Windows 버전:
WSL 버전:
WSL 배포판 이름:
WSL 모드(1 또는 2):
Ubuntu VERSION_ID:
Ubuntu 사용자 이름:
프로젝트 예정 경로:
VS Code의 WSL 연결 표시:
```

질문할 때 이 정보를 함께 제공하면 상대와 AI가 문제를 훨씬 빠르게 좁힐 수 있다.

자주 생기는 오해와 해결 방향

apt를 PowerShell에서 실행했다

PowerShell은 Linux의 apt Package 관리자를 알지 못한다. Ubuntu Terminal을 열거나 PowerShell에서 `wsl`을 입력해 기본 배포판 Shell로 들어간 뒤 실행한다. Prompt만 보고도 환경을 구별하는 습관이 중요하다.

wsl --status를 Ubuntu에서 실행했다

`wsl.exe`는 Windows가 WSL을 관리하는 명령이다. Ubuntu 안에서 Windows 실행 파일을 호출하는 방법도 있지만, 관리 명령은 PowerShell에서 실행한다고 원칙을 정하면 혼란이 줄어든다.

프로젝트가 /mnt/c 아래에 있다

간단한 Text 파일은 잘 열리더라도 Build가 느리거나 권한, Symbolic Link, 파일 감시에서 예상치 못한 문제가 생길 수 있다. ROS2 Workspace와 Git 저장소는 `/home/사용자이름` 아래에 두고 WSL 연결 VS Code로 편집한다.

VS Code Terminal에서 ros2를 찾지 못한다

먼저 창 왼쪽 아래에 WSL 연결 표시가 있는지, `pwd`가 `/home/...`인지 확인한다. Windows PowerShell Terminal을 연 상태라면 Ubuntu에 설치된 `ros2` 명령은 보이지 않는다. WSL 연결이 맞는데도 명령이 없다면 ROS2 설치와 환경 Source 여부를 3장에서 확인한다.

sudo 비밀번호가 Windows PIN과 다르다

Ubuntu 사용자 계정과 Windows 계정은 별개다. sudo는 Ubuntu를 처음 설정할 때 만든 Linux 사용자의 비밀번호를 요구한다. 입력 중에는 별표조차 표시되지 않지만 Keyboard 입력은 전달되고 있다. 비밀번호를 잊었다면 검증되지 않은 명령을 반복하기보다 Ubuntu 사용자 복구 절차를 확인해야 한다.

Linux에서 파일 이름의 대소문자가 달라 오류가 난다

Linux는 `Launch.py`, `launch.py`, `LAUNCH.py`를 서로 다른 이름으로 본다. Import나 실행 명령의 철자가 실제 파일 이름과 정확히 같은지 확인한다. Git에서 대소문자만 바꾼 파일 이름도 주의해서 다룬다.

GUI Application 창이 열리지 않는다

먼저 WSL 1이 아니라 WSL2인지 확인한다. WSL과 Graphics 지원을 Update한 뒤 모든 WSL Process를 종료하고 다시 시작할 수 있다. 그래도 해결되지 않으면 Windows Version, GPU Driver, 오류 메시지를 함께 기록한다. DISPLAY 환경 변수를 무작정 덮어쓰는 오래된 글은 현재 WSLg 환경과 맞지 않을 수 있다.

Native Ubuntu 사용자는 어떻게 따라갈까

컴퓨터에 Ubuntu 24.04를 직접 설치한 독자는 WSL 계층이 없다. Windows PowerShell과 `wsl` 명령도 필요하지 않다. Ubuntu Terminal에서 Package 설치, Git, VS Code, ROS2 명령을 모두 실행하면 된다.

Native Ubuntu의 권장 프로젝트 경로도 동일하게 `/home/사용자이름/...`이다. ROS2 Jazzy와 Gazebo Harmonic 실습 내용은 대부분 그대로 적용된다. 다만 GUI 표시, Network, USB Device 연결 절차에서는 WSL2와 차이가 있을 수 있으므로 책에서 환경별 표시를 확인한다.

AI와 함께 복습하고 검증하기

AI에게 “ROS가 안 돼”라고만 질문하면 가능한 원인이 너무 많다. AI는 부족한 정보를 추측하다가 Windows 명령과 Ubuntu 명령을 섞거나 다른 ROS2 Version의 해법을 제시할 수 있다. 좋은 질문에는 **환경, 현재 위치, 실행한 명령, 원문 오류, 기대한 결과**가 들어간다.

다음 Prompt를 자신의 결과에 맞게 채워 사용해 보자.

나는 Windows 11에서 WSL 2의 Ubuntu 24.04를 사용한다.
ROS2 배포판은 Jazzy이다.

현재 확인 결과:

- PowerShell의 ``wsl -l -v`` 결과: [여기에 붙여 넣기]
- Ubuntu의 ``cat /etc/os-release`` 결과: [여기에 붙여 넣기]
- Ubuntu의 ``pwd`` 결과: [여기에 붙여 넣기]
- VS Code 왼쪽 아래 표시: [여기에 적기]

실행한 명령:

[한 줄씩 정확히 붙여 넣기]

오류 원문:

[요약하지 말고 그대로 붙여 넣기]

기대한 결과:

[무엇을 하려 했는지 적기]

다음 형식으로 답해 줘.

1. 이 명령을 PowerShell과 Ubuntu 중 어디에서 실행해야 하는지
2. 가장 가능성 높은 원인 세 가지
3. 설정을 바꾸기 전에 실행할 읽기 전용 확인 명령
4. Ubuntu 24.04와 ROS2 Jazzy에 맞는 공식 문서 링크
5. 확실하지 않은 내용은 추측이라고 표시

AI의 답을 받으면 명령을 곧바로 모두 실행하지 않는다. 다음 순서로 검토한다.

1. 명령의 실행 위치가 분명한가?
2. 파일 삭제, 배포판 등록 해제, 재설치 같은 되돌리기 어려운 작업이 포함되어 있는가?
3. Humble, Foxy, ROS 1처럼 다른 Version의 설명이 섞이지 않았는가?
4. 먼저 상태를 읽는 명령으로 가설을 확인하게 하는가?
5. 공식 문서와 실제 출력으로 답을 검증할 수 있는가?

특히 `ws1 --unregister`는 배포판과 그 안의 Data를 제거할 수 있다. 단순한 Trouble Shooting 과정에서 의미를 모른 채 실행해서는 안 된다. AI에게도 파괴적인 명령을 제외하고 진단부터 제시하라고 요구하는 습관을 들이자.

AI Agent로 ROS2 설치하기

WSL2나 ROS2를 처음 설치할 때는 저장소 등록, GPG 키 설정, Package 의존성, 환경 변수처럼 낯선 작업에서 오류를 만날 수 있다.

최근에는 이러한 설치 작업 자체를 **AI Agent에게 맡기는 것도** 좋은 방법이다.

일반적인 ChatGPT와 대화하면서 명령어를 하나씩 복사하여 실행하는 방법도 있지만, Agent를 사용하면 AI가 직접 터미널 명령을 실행하고 결과를 확인한 뒤 오류가 발생하면 다시 원인을 분석하여 다음 명령을 실행할 수 있다.

즉,

명령어 생성 → 실행 → 오류 확인 → 수정 → 재실행 → 설치 확인

과정을 Agent가 상당 부분 대신 수행할 수 있다.



그림 2-8. Agent는 확인·설치·오류 분석·재실행·검증을 이어서 수행할 수 있지만, 시스템 변경의 최종 승인과 확인은 사용자가 맡아야 한다.

설치 작업에 사용할 수 있는 AI Agent

2026년 현재 Windows에서 사용할 수 있는 개발 Agent의 예는 다음과 같다.

Agent	Windows	WSL 사용	터미널 실행	특징
OpenAI Codex	◎	◎	◎	Windows + WSL 환경에 특히 편리
Cursor Agent	◎	◎	◎	IDE와 Agent를 함께 사용하기 좋음
GitHub Copilot Agent	◎	◎	◎	VS Code 사용자에게 편리
Google Antigravity	◎	◎	◎	Gemini 기반 Agent 환경
Claude Code	◎	◎	◎	터미널 중심의 Agent

이 책에서는 처음 Agent를 사용하는 사람이라면 **Codex**를 가장 먼저 추천한다.

Codex는 Windows에서 사용할 수 있으며 Windows에 설치된 WSL 배포판을 선택해서 작업할 수도 있다. 따라서 Windows 11 + WSL2 + Ubuntu 환경을 구축하거나 ROS2를 설치하는 것과 같은 작업에 잘 맞는다.

예를 들어 Codex에게 다음과 같이 요청할 수 있다.

나는 Windows 11에서 WSL2의 Ubuntu 24.04를 사용하려고 한다.

WSL2가 제대로 설치되어 있는지 먼저 확인하고,
Ubuntu 24.04가 설치되어 있지 않다면 설치해줘.

그 다음 ROS2 Jazzy를 설치하려고 한다.

ROS2 공식 문서를 확인해서 최신 설치 방법을 사용하고
ROS2 Jazzy Desktop을 설치해줘.

설치 과정에서 오류가 발생하면 오류 내용을 분석해서 해결하고
설치가 끝나면

```
ros2 --help
```

명령과 간단한 publisher/subscriber 예제를 실행해서
정상적으로 설치되었는지도 확인해줘.

중요한 시스템 설정을 변경하거나 삭제 명령을 실행해야 한다면
실행하기 전에 나에게 설명해줘.

Codex

Codex는 데스크톱 앱뿐 아니라 CLI와 IDE 확장 프로그램 형태로도 사용할 수 있다.
Windows 버전에서는 WSL 배포판을 선택하여 Linux 환경에서 작업할 수도 있다.

따라서 다음과 같이 요청할 수 있다.

현재 WSL 환경을 확인해줘.

Ubuntu 24.04가 설치되어 있는지 확인하고
ROS2 Jazzy 설치에 문제가 없는 환경인지 검사해줘.

문제가 없다면 ROS2 Jazzy Desktop을 설치하고
설치 후 talker/listener 예제를 실행해서 검증해줘.

Cursor Agent

Cursor는 VS Code와 비슷한 형태의 AI 기반 코드 편집기이다.

Cursor의 Agent는 프로젝트의 파일을 읽고 수정하는 것뿐 아니라 터미널 명령도 직접 실행할 수 있다. 필요한 dependency를 설치하거나 프로그램을 빌드하고 오류를 확인하여 수정하는 작업도 가능하다.

Windows용 프로그램이 제공되며 WSL에서도 Cursor CLI를 사용할 수 있다.

따라서 ROS2 프로젝트를 개발하면서

이 ROS2 workspace를 분석해줘.

필요한 dependency를 rosdep으로 설치하고
colcon build를 실행해줘.

빌드 오류가 발생하면 원인을 찾아 수정하고
다시 build해서 정상적으로 완료되는지 확인해줘.

처럼 요청하는 방식이 유용하다.

GitHub Copilot Agent

이미 VS Code를 사용하고 있다면 **GitHub Copilot의 Agent Mode**도 좋은 선택이다.

Agent Mode에서는 Copilot이 필요한 파일을 찾고 여러 파일을 수정하며 터미널 명령을 실행하고, 작업이 완료될 때까지 발생한 문제를 반복적으로 해결할 수 있다.

Google Antigravity

Google의 **Antigravity** 역시 Windows에서 사용할 수 있는 Agent 개발 환경이다.

Antigravity는 독립 실행형 환경뿐 아니라 VS Code 등의 IDE 확장과 CLI도 제공한다. Agent가 에디터와 터미널을 오가면서 여러 단계의 개발 작업을 수행할 수 있다.

Windows에서는 PowerShell을 통해 CLI를 설치하여 터미널 기반 Agent로 사용할 수도 있다.

따라서 Gemini 계열 모델을 선호한다면 Antigravity도 좋은 대안이 될 수 있다.

참고로 이름은 Gravity가 아니라 Google Antigravity이다.

어떤 Agent를 선택해야 할까?

처음 사용하는 사람이라면 여러 Agent를 모두 설치할 필요는 없다.

이미 VS Code와 GitHub Copilot을 사용하고 있다면 **Copilot Agent Mode**, VS Code와 비슷한 환경에서 Agent 중심으로 개발하고 싶다면 **Cursor**, Gemini 기반 환경을 사용하고 싶다면 **Google Antigravity**를 선택할 수 있다.

중요한 것은 어떤 Agent를 사용하는가보다 **Agent에게 작업의 목표와 환경을 정확하게 알려주는 것**이다.

예를 들어 단순히

ROS2 설치해줘.

라고 요청하는 것보다

Windows 11을 사용하고 있다.

WSL2에 Ubuntu 24.04를 설치하고
그 안에 ROS2 Jazzy Desktop을 설치하려고 한다.

반드시 ROS2 공식 문서를 먼저 확인하고
현재 운영체제와 ROS2 버전이 호환되는지 확인해줘.

설치 → 환경 설정 → rosdep 설정 → 설치 확인 →
talker/listener 실행까지 진행해줘.

오류가 발생하면 원인을 분석해서 해결한 후 계속 진행해줘.

처럼 실행 환경과 확인 절차를 함께 적는다.

Agent를 사용한다고 해서 모든 명령을 무조건 허용할 필요도 없다. `sudo`, 시스템 설정 변경, 파일 삭제와 같이 시스템에 영향을 줄 수 있는 작업은 실행 내용을 확인한 뒤 승인하는 습관을 들이는 것이 좋다.

AI Agent를 사용하더라도 Linux와 ROS2의 설치 과정을 이해해야 한다. Agent는 **설치 과정의 시행착오를 줄이고 ROS2 학습과 개발에 시간을 쓸 수 있도록 돕는 도구**다.

스스로 확인하기

1. WSL2와 Ubuntu를 같은 것으로 부르면 정확하지 않은 이유는 무엇인가?
 2. PS C:\Users\mina> Prompt에서 `sudo apt update`를 실행하면 안 되는 이유는 무엇인가?
 3. `/mnt/c/Users/mina/robot_ws`보다 `/home/mina/robot_ws`를 권장하는 이유를 두 가지 말해 보자.
 4. Windows용 VS Code 화면을 사용하면서 Ubuntu의 Git과 Python을 실행할 수 있는 구조를 설명해 보자.
 5. Ubuntu에서 실행한 RViz2 창이 Windows Desktop에 보일 수 있는 이유는 무엇인가?
 6. WSL2의 IP Address가 바뀔 수 있다는 사실이 나중에 실제 로봇 연결에서 왜 중요할까?
- 답을 외우기보다 그림 2-1을 보지 않고 자신의 말로 계층을 그려 보는 편이 좋다.

2장 마무리

실습 완료 체크리스트

- ☐ Windows와 Ubuntu Prompt를 눈으로 구별할 수 있다.
- ☐ PowerShell에서 `wsl -l -v`를 실행하고 WSL Version을 확인할 수 있다.
- ☐ Ubuntu에서 `cat /etc/os-release`로 24.04 여부를 확인할 수 있다.
- ☐ `/home`, `/mnt/c`, `C:\Users`의 관계를 설명할 수 있다.
- ☐ ROS2 프로젝트를 Linux Home Directory에 둘 이유를 설명할 수 있다.
- ☐ VS Code가 WSL에 연결되었는지 표시와 `pwd`로 확인할 수 있다.
- ☐ GUI 창의 표시 위치와 Process의 실행 위치가 다를 수 있음을 설명할 수 있다.
- ☐ 문제를 질문할 때 환경 정보와 오류 원문을 함께 제공할 수 있다.

핵심 정리

- Windows는 주 운영체제이고 WSL2는 실제 Linux Kernel을 사용하는 관리형 가상 환경을 제공한다.
- Ubuntu는 WSL2 위에서 사용하는 Linux 배포판이며 ROS2 Jazzy는 Ubuntu 위에 설치된다.
- WSL 설치와 상태 관리는 PowerShell에서, Ubuntu Package와 ROS2 작업은 Ubuntu Terminal에서 수행한다.
- Windows의 `C:\`는 Ubuntu에서 `/mnt/c`로 접근할 수 있지만 ROS2 프로젝트는 `/home` 아래에 두는 편이 좋다.
- Remote - WSL을 사용하면 Windows의 VS Code 화면과 Ubuntu 안의 파일·Git·Python·ROS2 도구를 자연스럽게 연결할 수 있다.
- WSLg는 Ubuntu의 Linux GUI Application 창을 Windows Desktop에 표시한다.
- WSL2는 Windows와 통합되어 있지만 File System과 Network가 완전히 같은 것은 아니다.
- 오류를 해결할 때는 Windows, WSL2, Ubuntu, ROS2 중 어느 계층의 문제인지 먼저 나눈다.

참고 자료

- Microsoft Learn, WSL 1과 WSL 2 비교
<https://learn.microsoft.com/windows/wsl/compare-versions>
- Microsoft Learn, WSL에서 여러 File System 사용하기
<https://learn.microsoft.com/windows/wsl/filesystems>
- Microsoft Learn, WSL에서 Linux GUI Application 실행하기
<https://learn.microsoft.com/windows/wsl/tutorials/gui-apps>
- Microsoft Learn, WSL 설치 방법
<https://learn.microsoft.com/windows/wsl/install>
- ROS2 Documentation, Ubuntu에서 ROS2 Jazzy 설치하기
<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>

다음 장 예고

환경의 관계와 명령을 입력할 위치를 구분했다. 3장에서는 Ubuntu 24.04의 상태를 점검하고 ROS2 Jazzy를 설치한다. 설치 명령을 단순히 복사하는 데 그치지 않고 Package 저장소, Locale, 환경 Source가 각각 왜 필요한지 확인하며 첫 `ros2` 명령 까지 실행한다.



마담 출판
ROBOTICS SERIES

ROS2 / 개발 환경과 첫 실행

Ubuntu 24.04와 ROS2 Jazzy 설치

03
CHAPTER

2장에서는 Windows, WSL2, Ubuntu, ROS2가 서로 다른 계층이라는 사실을 확인했다. 이제 Ubuntu 24.04 안에 ROS2 Jazzy를 설치하고, 새 Terminal에서 환경을 적용한 뒤 두 Node가 실제로 Message를 주고받는지 검증한다.

학습 목표

- ROS2 Jazzy와 Ubuntu 24.04를 함께 사용하는 이유는 무엇인가?
- Locale, Ubuntu universe, ROS2 APT 저장소는 각각 왜 필요한가?
- ros-jazzy-desktop과 ros-jazzy-ros-base는 무엇이 다른가?
- source /opt/ros/jazzy/setup.bash는 현재 Terminal에 무엇을 더하는가?
- 설치 성공을 환경 변수, Package 검색, Talker-Listener, ros2 doctor로 어떻게 검증하는가?
- ros2: command not found와 Package not found를 어떤 순서로 진단하는가?

이 장의 학습 지도

01 환경 확인

02 Jazzy 설치

03 송수신 검증

완료 기준

01 Ubuntu 24.04-UTF-8 Locale·설치 패키지를 확인하고, ROS_DISTRO가 jazzy인지 기록한다.

02 새 Terminal에서도 ROS2 환경이 적용되고 Talker-Listener-Turtlesim 실행 항목이 검색되는지 확인한다.

03 C++ Talker의 발행 문자열이 Python Listener에 연속 수신되는 것을 두 Terminal에서 비교한다.

04 ros2 doctor --report의 주요 결과를 저장하고, 통신 성공·경고·GUI 미검증 여부를 구분해 기록한다.

개념을 이해하고 · 직접 실행하고 · 결과로 확인하기

설치의 목표는 `apt install` 명령이 오류 없이 끝나는 것이 아니다. 다음 네 가지 근거가 모두 있어야 설치가 끝났다고 판단할 수 있다.

1. Ubuntu Version과 CPU Architecture가 Jazzy 지원 조건에 맞는다.
2. ROS2 Package와 실행 파일이 올바른 위치에 설치되었다.
3. 현재 Shell에 Jazzy 환경이 적용되었다.
4. C++ Talker가 발행한 Message를 Python Listener가 받는다.

이 장의 명령은 **Windows 11의 WSL2에서 실행하는 Ubuntu 24.04 LTS**를 기준으로 한다. Native Ubuntu 24.04에서도 PowerShell과 WSL 확인 단계를 제외하면 같은 절차를 사용할 수 있다.

ROS2 설치의 Ubuntu Terminal에서 진행한다. Prompt가 `PS C:\...>`로 시작한다면 PowerShell이므로 설치 명령을 입력하지 않는다.

이 장을 마치면 다음 질문에 답할 수 있다.

이 장에서는 공식 Debian Package를 이용해 설치한다. ROS2 자체를 소스 코드부터 Build하는 방법이나 Docker Image를 사용하는 방법은 다루지 않는다. 입문자에게는 의존성을 함께 관리하고 Ubuntu Update를 통해 유지보수할 수 있는 Debian Package 방식이 가장 재현하기 쉽다.

Jazzy Jalisco를 선택하는 이유

ROS2의 배포판에는 Jazzy, Kilted, Lyrical처럼 이름과 지원 기간이 있다. 인터넷에서 가장 새로운 배포판을 발견했다고 해서 이 책의 명령에서 jazzy만 다른 이름으로 바꾸면 되는 것은 아니다. Ubuntu Version, Python Version, Gazebo 조합과 Package 제공 상태가 배포판마다 다르기 때문이다.

이 책은 다음 조합을 하나의 기준으로 사용한다.

- Ubuntu 24.04 LTS, 코드명 Noble
- ROS2 Jazzy Jalisco
- Python 3.12 계열의 Ubuntu System Python
- ROS2와 연결할 때 Gazebo Harmonic

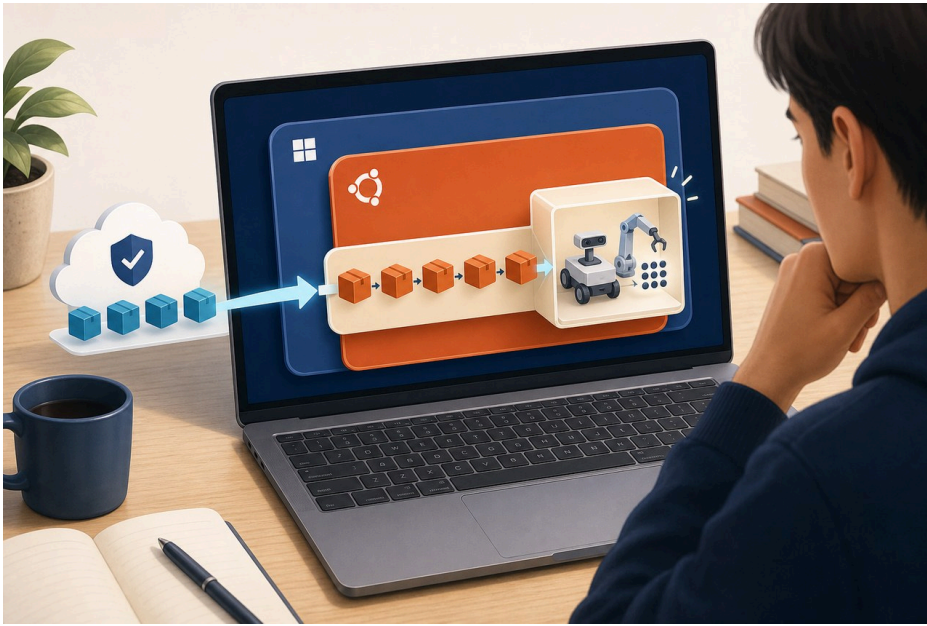


그림 3-1. 이 책에서는 Windows의 WSL2 안에서 Ubuntu 24.04를 실행하고, 공식 APT 저장소의 Package로 ROS2 Jazzy 개발 환경을 구성한다.

Jazzy Jalisco는 2024년 5월 23일 공개된 ROS2의 **열 번째 릴리스**다. Ubuntu 24.04의 amd64와 arm64가 Tier 1 Platform이며 지원 종료 예정일은 2029년 5월이다. 이 책에서는 공식 Release 문서의 번호와 지원 일정을 기준으로 한다.

2026년에 더 새로운 ROS2 배포판이 존재하더라도 Jazzy가 곧바로 사용할 수 없는 Version이 되는 것은 아니다. 이 책의 목표는 매년 API와 명령을 바꾸는 것이 아니라, 수업 자료·소스 코드·실습 환경이 함께 검증된 하나의 기준을 제공하는 것이다.

설치 전에 결정할 것

Desktop과 ROS Base

APT 저장소에는 용도에 따라 묶어 놓은 Meta Package가 있다.

- **Desktop Install: `ros-jazzy-desktop`**
 - ROS2 Core, RViz2, Demo, Tutorial 등 입문 실습에 필요한 도구를 함께 설치한다.
 - 이 책의 권장 선택이다.
- **ROS Base Install: `ros-jazzy-ros-base`**
 - 통신 Library, Message Package, CLI 같은 최소 구성만 설치한다.
 - GUI가 필요 없는 Server나 용량을 줄여야 하는 장치에 적합하다.
- **Development Tools: `ros-dev-tools`**
 - colcon, rosdep 등 Package 개발에 필요한 도구를 제공한다.
 - 뒤에서 직접 ROS2 Package를 만들 것이므로 함께 설치한다.

desktop과 ros-base를 둘 다 설치할 필요는 없다. desktop은 ros-base의 기능을 포함하는 더 큰 구성이다. 이 책에서는 다음 두 Package를 설치한다.

```
sudo apt install ros-jazzy-desktop ros-dev-tools
```

System Python을 유지한다

Ubuntu의 ROS2 Debian Package는 해당 Ubuntu 배포판의 System Python과 맞춰 Build된다. 이 책의 기준 환경에서는 `python3 --version`이 Python 3.12 계열을 보여 준다.

Conda나 다른 Python을 Shell 전체의 기본 Interpreter로 바꾸면 `rclpy` 같은 Binary Module과 Version이 맞지 않을 수 있다. 입문 실습에서는 Ubuntu의 `/usr/bin/python3`를 사용한다.

설치 전 상태를 기록한다

먼저 읽기 전용 명령으로 현재 상태를 확인한다. 이미 설치된 환경을 다시 덮어쓰지 않고, 실패했을 때 어느 단계까지 정상이었는지 비교하기 위해서다.

PowerShell에서 WSL 확인

Windows PowerShell에서 다음 명령을 실행한다.

```
wsl --status
wsl -l -v
```

다음과 비슷하게 Ubuntu-24.04와 VERSION 2가 보여야 한다.

NAME	STATE	VERSION
* Ubuntu-24.04	Running	2

배포판 이름이 조금 달라도 Ubuntu 24.04이고 WSL2라면 계속할 수 있다. Ubuntu 22.04에 Jazzy용 명령을 억지로 적용하거나, 기존 배포판을 지우고 다시 만드는 방식으로 해결하지 않는다.

Ubuntu 24.04가 아직 없을 때만 관리자 PowerShell에서 설치한다.

```
wsl --install -d Ubuntu-24.04
```

Windows가 재부팅을 요구하면 열려 있는 작업을 저장하고 안내에 따른다. 처음 Ubuntu를 실행할 때 만드는 Linux 사용자 이름과 비밀번호는 Windows 계정과 별개다.

Ubuntu에서 운영체제와 Architecture 확인

이후 명령은 Ubuntu Terminal에서 실행한다.

```
cat /etc/os-release
dpkg --print-architecture
```

```
uname -m
python3 --version
```

핵심 판정 기준은 다음과 같다.

- /etc/os-release에 VERSION_ID="24.04"와 VERSION_CODENAME=noble이 있다.
- dpkg --print-architecture가 일반 PC에서는 amd64, 64-bit ARM 장치에서는 arm64다.
- python3 --version이 기준 환경에서는 Python 3.12 계열이다.

uname -m은 같은 Architecture를 다른 이름으로 표현할 수 있다. 예를 들어 amd64 장치에서 x86_64가 출력되는 것은 정상이다.

기존 ROS2 설치 확인

```
test -f /opt/ros/jazzy/setup.bash && echo "Jazzy setup file exists"
dpkg -l | grep -E '^ii[[:space:]]+ros-jazzy-(desktop|ros-base)'
command -v ros2
```

세 명령의 의미는 서로 다르다.

- 첫 번째 명령은 Jazzy 환경 설정 파일의 존재를 확인한다.
- 두 번째 명령은 APT가 관리하는 Meta Package의 설치 상태를 확인한다.
- 세 번째 명령은 **현재 Shell**에서 ros2 실행 파일을 찾을 수 있는지 확인한다.

/opt/ros/jazzy/setup.bash는 있는데 command -v ros2 결과가 비어 있다면 재설치보다 환경 Source를 먼저 확인해야 한다. 반대로 파일 자체가 없다면 설치 단계가 필요하다.

다음 항목을 학습 기록에 적어 둔다.

```
확인 날짜:
WSL 배포판 이름과 Version:
Ubuntu VERSION_ID와 Codename:
CPU Architecture:
Python Version:
/opt/ros/jazzy/setup.bash 존재 여부:
기존 ros2 명령 검색 결과:
```

1단계: UTF-8 Locale 확인

Locale은 프로그램이 언어, 문자 Encoding, 날짜와 숫자 표기 방식을 해석하는 규칙이다.

ROS2 공식 설치 문서는 UTF-8을 지원하는 Locale을 요구한다. 한국어 파일 이름과 Log를 다룰 때도 UTF-8 여부가 중요하다.

```
locale
locale charmap
```

locale charmap 결과가 UTF-8이면 이 조건은 이미 충족된 것이다. C.UTF-8, en_US.UTF-8, ko_KR.UTF-8처럼 이름이 다르더라도 UTF-8을 지원하면 사용할 수 있다. 최소 설치 환경에서 UTF-8 Locale이 없다면 공식 문서의 다음 절차를 사용한다.

```
sudo apt update
sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8
locale
```

마지막 locale은 변경 명령의 일부가 아니라 결과를 검증하는 명령이다. 기존에 정상적인 UTF-8 Locale을 사용하고 있다면 무조건 en_US.UTF-8로 바꿀 필요는 없다.

2단계: Ubuntu Universe 저장소 활성화

Ubuntu의 APT는 여러 Software 저장소에서 Package 목록을 받아 온다. ROS2 Package가 의존하는 일부 Ubuntu Package를 찾으려면 universe 저장소가 활성화되어 있어야 한다.

```
sudo apt update
sudo apt install software-properties-common
sudo add-apt-repository universe
```

universe가 이미 활성화되어 있다는 안내는 오류가 아니다. 상태는 다음처럼 확인할 수 있다.

```
apt-cache policy | grep universe
```

3단계: ROS2 APT 저장소 등록

Ubuntu 기본 저장소만으로는 공식 ROS2 Jazzy Debian Package를 찾을 수 없다. ROS2 저장소의 위치와 서명 Key를 APT에 알려 주어야 한다.

예전 글에서는 GPG Key 파일과 ros2.list를 직접 만드는 명령을 볼 수 있다. 현재 공식 문서는 저장소 설정을 관리하는 **ros2-apt-source Package** 설치를 안내한다. 이 Package를 사용하면 저장소 설정이 바뀔 때 APT를 통해 함께 Update할 수 있다.

```
sudo apt update
sudo apt install curl -y
```

```
export ROS_APT_SOURCE_VERSION=$(curl -s \
  https://api.github.com/repos/ros-infrastructure/ros-apt-
source/releases/latest \
  | grep -F "tag_name" \
  | awk -F'"' '{print $4}')

curl -L -o /tmp/ros2-apt-source.deb \
  "https://github.com/ros-infrastructure/ros-apt-
source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-
source_${ROS_APT_SOURCE_VERSION}.${. /etc/os-release && echo
${UBUNTU_CODENAME:-${VERSION_CODENAME}})_all.deb"

sudo dpkg -i /tmp/ros2-apt-source.deb
```

긴 curl 명령은 줄 끝의 역슬래시 \까지 포함해 한 Block으로 실행한다. 역슬래시 뒤에는 공백을 넣지 않는다.

설치 결과를 확인한다.

```
dpkg-query -W ros2-apt-source
sudo apt update
apt-cache policy ros-jazzy-desktop
```

첫 번째 명령은 ros2-apt-source Version을 보여 준다. 마지막 명령에서 ros-jazzy-desktop의 Candidate Version이 보이면 APT가 ROS2 Package 목록을 찾을 수 있는 상태다.

Download가 실패했다면 dpkg -i를 계속 실행하지 않는다. HTML 오류 Page를 .deb라는 이름으로 저장했을 수도 있다.

```
echo "$ROS_APT_SOURCE_VERSION"
ls -lh /tmp/ros2-apt-source.deb
file /tmp/ros2-apt-source.deb
```

4단계: Ubuntu Package Update

새 저장소를 등록했으므로 Package 목록을 다시 받고, 공식 안내에 따라 설치 전에 Ubuntu Package를 최신 상태로 맞춘다.

```
sudo apt update
sudo apt upgrade
```

apt update는 Package 목록을 갱신하고 apt upgrade는 실제 설치 Package를 갱신한다. apt upgrade는 ROS2만이 아니라 여러 Ubuntu Package를 바꿀 수 있다. 수업 직전의

공용 PC나 중요한 작업이 실행 중인 장비에서는 변경 목록과 재시작 필요 여부를 먼저 확인하고 충분한 시간을 확보한다.

의존성 오류가 나타났을 때 무조건 `--fix-broken`, 저장소 파일 삭제, 전체 재설치를 반복하지 않는다. `sudo apt update`의 첫 오류부터 원문을 보관하고 Ubuntu Codename과 저장소 등록 상태를 확인한다.

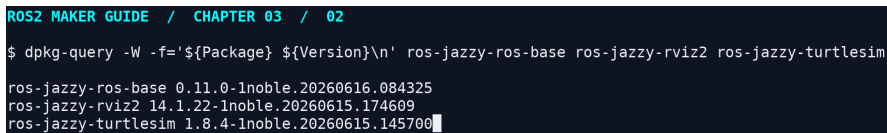
5단계: ROS2 Desktop과 개발 도구 설치

```
sudo apt install ros-jazzy-desktop ros-dev-tools
```

설치 전 APT가 보여 주는 새 Package 수, Download 용량, 제거되는 Package가 있는지를 확인한다. 설치가 끝나면 파일과 Package 상태를 검증한다.

```
test -f /opt/ros/jazzy/setup.bash && echo "setup file: OK"
dpkg-query -W ros-jazzy-desktop ros-dev-tools
ls /opt/ros/jazzy
```

`/opt/ros/jazzy`에는 `bin`, `lib`, `share`, `setup.bash` 같은 항목이 보인다. 이 디렉터리는 APT로 설치한 ROS2 Jazzy Package의 기본 설치 공간이다. APT가 관리하는 파일을 임의로 수정하거나 삭제하지 않는다. 직접 작성하거나 Git으로 내려받아 빌드하는 소스 코드는 이곳이 아니라 홈 디렉터리 아래의 `~/ros_ws/src`에 둔다.



```
ROS2 MAKER GUIDE / CHAPTER 03 / 02
$ dpkg-query -W -f='${Package} ${Version}\n' ros-jazzy-ros-base ros-jazzy-rviz2 ros-jazzy-turtlesim
ros-jazzy-ros-base 0.11.0-1noble.20260616.084325
ros-jazzy-rviz2 14.1.22-1noble.20260615.174609
ros-jazzy-turtlesim 1.8.4-1noble.20260615.145700
```

그림 3-2. 설치된 ROS2 Package 버전 조회 결과. 설치 명령의 입력과 실제 설치 상태를 구분한다.

6단계: 현재 Terminal에 ROS2 환경 적용

Package가 디스크에 설치되었다고 해서 모든 Terminal이 자동으로 `ros2`의 위치와 Package 목록을 아는 것은 아니다. 다음 명령은 Jazzy 설치가 제공하는 Shell 설정을 현재 Terminal Process에 읽어 들인다.

```
source /opt/ros/jazzy/setup.bash
```

`source`는 새 프로그램을 설치하는 명령이 아니다. `PATH`, `PYTHONPATH`, `AMENT_PREFIX_PATH`, `ROS_DISTRO`처럼 ROS2 실행과 Package 탐색에 필요한 환경을 현재 Shell에 추가한다.

```
echo "$ROS_DISTRO"
echo "$ROS_VERSION"
echo "$ROS_PYTHON_VERSION"
command -v ros2
ros2 --help
```

기준 출력은 다음과 같다.

- ROS_DISTRO → jazzy
- ROS_VERSION → 2
- ROS_PYTHON_VERSION → 3
- command -v ros2 → /opt/ros/jazzy/bin/ros2
- ros2 --help → ROS2 CLI 사용법

source가 아무 Text도 출력하지 않는 것은 정상이다. 결과는 환경 변수와 다음 명령의 동작으로 확인한다.



그림 3-3. setup.bash를 Source하면 현재 Terminal에서 ROS2 실행 파일, Python Module, Package 경로를 찾을 수 있다. 별도로 연 새 Terminal에는 다시 환경을 적용해야 한다.

Underlay와 Overlay의 첫 모습

/opt/ros/jazzy/setup.bash로 적용한 공식 설치 공간을 **Underlay**라고 생각할 수 있다. 뒤에서 이 저장소의 gong_ws를 Build하면 다음처럼 두 환경을 순서대로 적용한다.

```
source /opt/ros/jazzy/setup.bash
source ~/ros_ws/install/local_setup.bash
```

첫 줄은 공식 Jazzy Package를, 둘째 줄은 학생이 Build한 Package를 현재 Terminal에 더한다. 아직 `ros_ws/install/local_setup.bash`가 없다면 Build 전이므로 둘째 줄을 실행하지 않는다. Workspace와 Overlay는 6장에서 자세히 다룬다.

```
ROS2 MAKER GUIDE / CHAPTER 03 / 01

$ printenv ROS_DISTRO; lsb_release -ds; python3 --version

jazzy
Ubuntu 24.04.4 LTS
Python 3.12.3
```

그림 3-4. ROS_DISTRO와 Ubuntu·Python 버전을 조회한 결과. Jazzy 환경이 적용되었는지 확인한다.

7단계: 새 Terminal에서 자동 적용

Bash는 새 Interactive Shell을 열 때 `~/.bashrc`를 읽는다. 여기에 Source 명령을 한 번만 등록하면 반복 입력을 줄일 수 있다.

먼저 중복 여부를 확인한다.

```
grep -nF 'source /opt/ros/jazzy/setup.bash' ~/.bashrc
```

아무 줄도 나오지 않을 때만 추가한다.

```
echo 'source /opt/ros/jazzy/setup.bash' >> ~/.bashrc
source ~/.bashrc
```

새 Ubuntu Terminal을 열어 다시 확인한다.

```
echo "$ROS_DISTRO"
command -v ros2
```

같은 Source 줄을 여러 번 추가하면 나중에 여러 배포판이나 Workspace를 함께 사용할 때 환경 순서를 이해하기 어려워진다. `~/.bashrc`를 수정하기 전에는 `grep`, 수정 후에는 새 Terminal에서 결과 확인이라는 습관을 들인다.

Alias는 설치가 아니다

수업 자료에는 반복 명령을 줄이기 위한 Alias 예가 있다.

```
alias sb='source ~/.bashrc'
alias cb='cd ~/ros_ws && colcon build --symlink-install'
```

Alias는 긴 명령에 짧은 별명을 붙일 뿐이다. ROS2를 설치하거나 환경을 대신 설정하지 않는다. 처음에는 원래 명령을 이해한 뒤 사용하고, 경로가 실제 자신의 저장소와 맞는지 확인한다. 특히 cb는 Build를 실행하므로 단순한 Directory 이동 Alias와 다르다.

8단계: C++ Talker와 Python Listener 통신 검증

설치 파일과 환경 변수가 보인다고 해서 통신까지 확인된 것은 아니다. 수업 자료의 3교시 실습처럼 두 Terminal을 사용해 Publisher와 Subscriber를 실행한다.

Terminal A: Talker 실행

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

```
[INFO] [...] [talker]: Publishing: 'Hello World: 1'
[INFO] [...] [talker]: Publishing: 'Hello World: 2'
```

Talker는 C++ Client Library를 사용해 /chatter Topic으로 Text Message를 발행한다.

Terminal B: Listener 실행

새 Ubuntu Terminal을 열고 실행한다.

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

```
[INFO] [...] [listener]: I heard: [Hello World: 1]
[INFO] [...] [listener]: I heard: [Hello World: 2]
```

이 실습은 ros2 run의 Package 검색, C++ rclcpp, Python rclpy, DDS/RMW Discovery, /chatter Topic의 Interface 일치, 실제 Message 전달을 함께 검증한다. 각 Terminal에서 Ctrl+C를 눌러 Node를 종료한다.

설치 성공을 단순히 “글자가 나왔다”라고 기록하지 말고 다음처럼 설명해 보자.

Terminal A의 /talker Node가 /chatter Topic으로
std_msgs/msg/String Message를 Publish했다.

Terminal B의 /listener Node가 같은 Topic과 Message Type을
Subscribe하여 'I heard' Log로 출력했다.

Node와 Topic의 전체 관찰 방법은 5장에서 다룬다. 지금은 서로 다른 언어로 작성된 두 실행 프로그램이 같은 ROS2 Interface 약속으로 통신했다는 점이 중요하다.

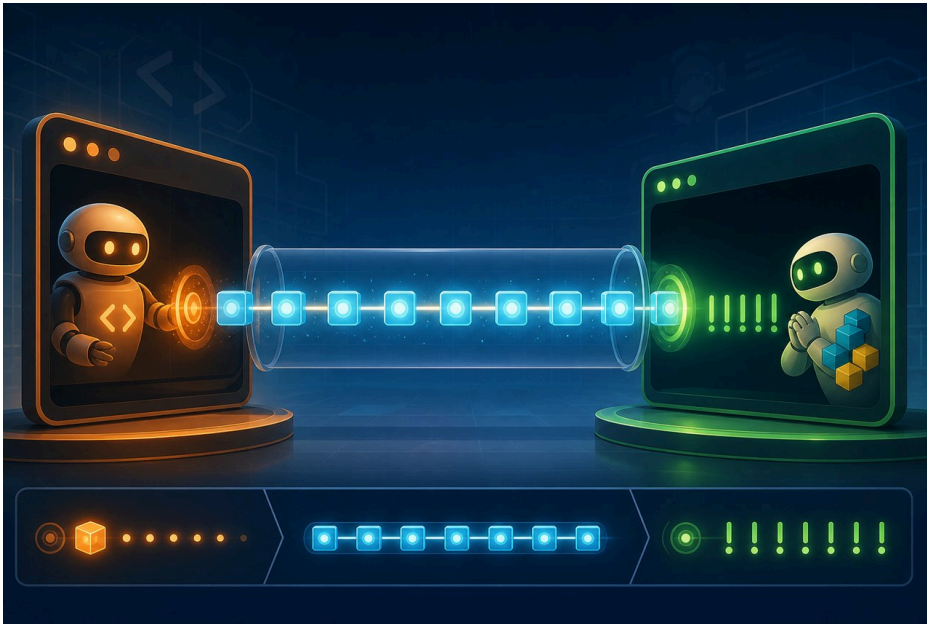


그림 3-5. Talker가 /chatter Topic으로 발행한 Message가 통신 채널을 지나 Listener에 도착하면 C++와 Python API, DDS Discovery, Interface 일치를 함께 검증할 수 있다.

9단계: Package와 System 보고서 확인

실행 가능한 Package 검색

```
ros2 pkg executables demo_nodes_cpp | grep talker
ros2 pkg executables demo_nodes_py | grep listener
ros2 pkg executables turtlesim
```

Turtlesim에서는 다음 네 항목이 검색되는 것이 기준이다.

```
turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
```

Package가 없을 때만 보충한다.

```
sudo apt update
sudo apt install \
  ros-jazzy-turtlesim \
  ros-jazzy-rqt \
  ros-jazzy-rqt-common-plugins
```

apt와 ros2 pkg는 서로 다른 질문에 답한다.

```
dpkg-query -W ros-jazzy-turtlesim
ros2 pkg executables turtlesim
```

- dpkg-query는 Ubuntu Package가 설치되었는가를 확인한다.
- ros2 pkg executables는 현재 ROS2 환경에서 실행 항목을 찾을 수 있는가를 확인한다.

ros2 doctor 실행

```
ros2 doctor --report
```

보고서에서 Distribution 이름이 jazzy인지, Platform이 Ubuntu 24.04와 WSL2 Kernel을 가리키는지, Middleware/RMW와 Network Interface 정보가 표시되는지 찾는다.

ros2 doctor --report는 상태를 수집하는 진단 도구이지 모든 문제를 자동으로 고치는 명령은 아니다. Warning이 하나 있다고 설치 전체가 실패한 것도 아니다. Talker-Listener 통신, Package 검색, 실제 오류를 함께 보고 판단한다.

기본 RMW는 일반적으로 Fast DDS지만 환경 변수나 추가 Package로 다른 구현을 선택할 수 있다. 이 장에서는 기본값을 임의로 바꾸지 않는다. WSL Network와 DDS Discovery는 11장에서 더 자세히 다룬다.

ROS2 MAKER GUIDE / CHAPTER 03 / 03

```
$ ros2 pkg executables turtlesim

turtlesim draw_square
turtlesim mimic
turtlesim turtle_teleop_key
turtlesim turtlesim_node
```

그림 3-6. Turtlesim의 설치된 실행 파일 목록. Node 실행 전에 Package를 찾을 수 있는지 확인한다.

선택 단계: rosdep 준비

rosdep은 package.xml에 선언된 의존성을 읽고 운영체제 Package로 연결해 주는 도구다. 6장 이후 Source Package를 Build할 때 사용한다.

```
command -v rosdep
test -f /etc/ros/rosdep/sources.list.d/20-default.list \
```

```
&& echo "rosdep sources already initialized"
```

System Source 목록이 아직 없을 때만 한 번 초기화한다.

```
sudo rosdep init
rosdep update
```

`sudo rosdep init`은 System 설정을 만드는 최초 한 번의 작업이고, `rosdep update`는 현재 사용자의 Cache를 갱신하는 반복 작업이다. `rosdep init`에서 이미 초기화되었다는 메시지가 나오면 같은 명령을 반복하지 않는다.

한 번에 확인하는 최소 검증 절차

설치 후 새 Ubuntu Terminal에서 다음 순서로 실행한다.

```
cat /etc/os-release | grep -E 'VERSION_ID|VERSION_CODENAME'
python3 --version

source /opt/ros/jazzy/setup.bash
echo "$ROS_DISTRO"
command -v ros2

dpkg-query -W ros-jazzy-desktop ros-dev-tools
ros2 pkg executables demo_nodes_cpp | grep talker
ros2 pkg executables demo_nodes_py | grep listener
ros2 pkg executables turtlesim

ros2 doctor --report
```

명령을 한꺼번에 붙여 넣기보다 한 Block씩 실행하고 결과를 기록한다. 긴 `doctor` 보고서는 다음처럼 화면과 파일에 함께 남길 수 있다.

```
mkdir -p ~/ros2-install-check
ros2 doctor --report | tee ~/ros2-install-check/ros2-doctor-
report.txt
```

비밀번호, Token, 사설 IP처럼 공유하지 말아야 할 정보가 있는지 확인한 뒤 보고서를 다른 사람이나 AI에게 전달한다.

자주 발생하는 오류와 해결 방법

ros2: command not found

```
ls -l /opt/ros/jazzy/setup.bash
source /opt/ros/jazzy/setup.bash
echo "$ROS_DISTRO"
command -v ros2
```

setup.bash가 있고 Source 후 명령이 보이면 설치 문제가 아니라 환경 적용 문제다. 파일이 없다면 ros-jazzy-desktop 설치 상태를 확인한다. PowerShell이나 Windows용 VS Code Terminal이라면 Ubuntu Terminal로 이동한다.

Package '...' not found

```
echo "$ROS_DISTRO"
ros2 pkg list | grep '^turtlesim$'
dpkg-query -W ros-jazzy-turtlesim
```

현재 Shell이 Jazzy를 Source했는지, ROS Package Index에 보이는지, Ubuntu Package가 설치되었는지를 순서대로 나눈다. 철자와 ros-jazzy-... 배포판 Prefix도 확인한다.

Unable to locate package ros-jazzy-desktop

```
cat /etc/os-release
dpkg-query -W ros2-apt-source
sudo apt update
apt-cache policy ros-jazzy-desktop
```

Ubuntu 24.04인지, ros2-apt-source가 설치되었는지, apt update의 저장소 오류가 없었는지 확인한다. apt update가 실패했는데 마지막 줄만 보고 설치를 계속하면 원인을 놓치기 쉽다.

Talker는 실행되지만 Listener가 받지 못한다

두 Terminal에서 공통으로 확인한다.

```
echo "$ROS_DISTRO"
echo "$ROS_DOMAIN_ID"
echo "$RMW_IMPLEMENTATION"
printenv ROS_AUTOMATIC_DISCOVERY_RANGE
ros2 node list
ros2 topic list
```

두 Node가 같은 ROS Domain과 호환되는 RMW/Discovery 설정을 사용해야 한다. 처음 설치를 검증할 때는 두 Node를 같은 WSL Ubuntu 안에서 실행하여 Windows Firewall과 외부 Network 변수를 줄인다. 기존 Robot 연결용 환경 변수가 있다면 함부로 삭제하지 말고 두 Terminal의 차이를 먼저 비교한다.

ros2 doctor가 Warning을 보여 준다

Warning 전체 문장을 읽고 설치나 환경 누락, Package Update 알림, Network 정보 중 어디에 해당하는지 구분한다. Talker-Listener가 통신하고 필요한 Package가 검색되는데 Update Warning만 있다면 설치 실패라고 단정하지 않는다. 반대로 doctor가 조용해도 실제 통신을 실행하지 않았다면 Runtime 검증은 끝난 것이 아니다.

GUI 창이 열리지 않는다

먼저 CLI 통신 검증과 GUI 표시 문제를 분리한다. Talker-Listener가 정상이라면 ROS2 Core 설치는 동작할 수 있다. 작업을 저장한 뒤 PowerShell에서 다음을 실행할 수 있다.

```
wsl --update
wsl --shutdown
```

wsl --shutdown은 실행 중인 모든 WSL 배포판을 종료하므로 Ubuntu Process와 편집 중인 파일을 먼저 정리한다. 현재 WSLg 환경에서 오래된 글의 DISPLAY 값을 무작정 덮어쓰지 않는다.

설치 결과를 검증 수준으로 기록하기

“설치 완료”라는 말 대신 확인한 수준을 나누어 기록한다.

- **정적 확인**
 - /opt/ros/jazzy/setup.bash가 존재한다.
 - APT에서 ros-jazzy-desktop, ros-dev-tools가 설치 상태다.
- **Shell 환경 확인**
 - ROS_DISTRO=jazzy다.
 - ros2 경로가 /opt/ros/jazzy/bin/ros2다.
- **Package 검색 확인**
 - Demo와 Turtlesim 실행 항목이 검색된다.
- **Runtime 통신 확인**
 - C++ Talker의 Message를 Python Listener가 받았다.
- **GUI 확인**
 - Turtlesim이나 RQt 창이 WSLg를 통해 열렸다.

앞 단계가 통과했다고 뒤 단계까지 자동으로 증명되는 것은 아니다. 예를 들어 Package 파일의 존재는 GUI 창이 정상 표시된다는 증거가 아니다. 이 구분은 뒤에서 Robot, Camera, Gazebo를 시험할 때도 계속 사용한다.

AI와 함께 복습하고 검증하기

AI에게 설치를 맡길 때도 목표와 허용 범위를 분명히 해야 한다. 특히 `sudo`, 저장소 변경, `Package Upgrade`는 System 상태를 바꾼다. 먼저 읽기 전용 확인을 수행하고, 변경할 항목과 검증 방법을 설명하게 한다.

Windows 11의 WSL 2에서 Ubuntu 24.04를 사용하고 있다.
ROS2 Jazzy Desktop을 공식 Debian Package 방식으로 설치하려고 한다.

먼저 WSL Version, Ubuntu Codename, CPU Architecture, UTF-8 Locale,
기존 `/opt/ros/jazzy`와 `ros2-apt-source` 설치 여부,
.bashrc의 Jazzy source 중복 여부를 읽기 전용 명령으로 확인해 줘.

현재 ROS2 Jazzy 공식 설치 문서와 대조하고,
변경하기 전에 실행 명령, 변경 대상, 예상 결과를 설명해 줘.

설치 후에는 다음을 각각 검증해 줘.

1. ROS_DISTRO와 `ros2` 실행 경로
2. `ros-jazzy-desktop`과 `ros-dev-tools` 설치 상태
3. C++ talker와 Python listener의 실제 송수신
4. `turtlesim` 실행 항목
5. `ros2 doctor --report`

다른 ROS2 배포판의 명령을 섞지 말고,
파일 삭제·배포판 제거·재부팅이 필요하면 실행 전에 이유를 알려 줘.
확인하지 못한 GUI나 외부 Robot 통신은 미검증이라고 표시해 줘.

AI의 답이 Ubuntu 24.04와 Jazzy 조합을 확인했는지, 현재 `ros2-apt-source` 절차를 쓰는 지, 기존 설치를 먼저 읽는지, `apt update`와 `apt upgrade`의 영향을 구분하는지, `Package` 설치와 Runtime 통신을 별도로 검증하는지 확인한다.

AI Agent가 Terminal을 직접 실행할 수 있어도 `sudo` 비밀번호 입력, System Upgrade, WSL 종료 같은 결정의 책임까지 사라지는 것은 아니다. Agent는 실행과 관찰을 돕는 도구이고, 사용자는 무엇이 바뀌는지 이해하고 결과를 검증해야 한다.

스스로 확인하기

1. Ubuntu 22.04에서 Package 이름의 jazzy만 바꾸어 설치하면 안 되는 이유는 무엇인가?
2. `apt update`와 `apt upgrade`는 각각 무엇을 바꾸는가?
3. Ubuntu universe와 ROS2 APT 저장소는 왜 둘 다 필요한가?
4. `/opt/ros/jazzy/setup.bash`가 있는데 `ros2` 명령이 보이지 않을 때 무엇부터 확인해야 하는가?
5. `ros-jazzy-desktop`과 `ros-dev-tools`의 역할은 어떻게 다른가?
6. Talker-Listener 실습이 단순한 파일 존재 확인보다 강한 검증인 이유는 무엇인가?
7. `dpkg-query -W ros-jazzy-turtlesim`과 `ros2 pkg executables turtlesim`은 무엇이 다른가?

8. ~/.bashrc에 Source 줄을 추가하기 전에 grep으로 확인해야 하는 이유는 무엇인가?
9. ros2 doctor --report의 Warning 하나만으로 설치 실패라고 단정하면 안 되는 이유는 무엇인가?

3장 마무리

실습 완료 체크리스트

- ☐ PowerShell에서 Ubuntu 24.04가 WSL2로 실행되는 것을 확인할 수 있다.
- ☐ Ubuntu에서 Version, Codename, Architecture, Python Version을 기록할 수 있다.
- ☐ UTF-8 Locale을 확인할 수 있다.
- ☐ universe와 ROS2 APT 저장소를 공식 절차로 준비할 수 있다.
- ☐ ros-jazzy-desktop과 ros-dev-tools의 설치 상태를 확인할 수 있다.
- ☐ source /opt/ros/jazzy/setup.bash의 의미를 설명할 수 있다.
- ☐ ROS_DISTRO=jazzy와 /opt/ros/jazzy/bin/ros2를 확인할 수 있다.
- ☐ ~/.bashrc에 같은 Source 줄이 중복되지 않았는지 확인할 수 있다.
- ☐ C++ Talker와 Python Listener의 실제 송수신을 확인할 수 있다.
- ☐ Turtlesim 실행 항목 네 개를 조회할 수 있다.
- ☐ ros2 doctor --report를 실행하고 핵심 항목을 기록할 수 있다.
- ☐ 정적·Shell·Package·Runtime·GUI 검증을 구분해 기록할 수 있다.

핵심 정리

- ROS2 Jazzy는 Ubuntu 24.04 Noble의 amd64와 arm64를 Tier 1 Platform으로 지원하며 2029년 5월까지 지원될 예정이다.
- Jazzy는 ROS2의 열 번째 릴리스다. 수업 자료의 “13번째” 표현은 공식 Release 자료와 맞지 않는다.
- 이 책은 ros-jazzy-desktop과 개발 도구인 ros-dev-tools를 사용한다.
- 설치 전에는 Ubuntu Version, Architecture, Locale, 기존 설치를 먼저 읽어 볼 필요한 재설치를 피한다.
- 현재 공식 절차는 ros2-apt-source Package로 ROS2 저장소와 Key 설정을 관리한다.
- source /opt/ros/jazzy/setup.bash는 설치가 아니라 현재 Shell에 ROS2 환경을 적용하는 작업이다.
- ~/.bashrc에는 Source 줄을 한 번만 등록하고 새 Terminal에서 결과를 검증한다.

- 설치 성공은 Package 존재뿐 아니라 C++ Talker와 Python Listener의 실제 통신으로 확인한다.
- 정적 확인, Shell 환경, Package 검색, Runtime 통신, GUI 실행은 서로 다른 검증 수준이다.

참고 자료

- ROS2 Documentation, Ubuntu에서 ROS2 Jazzy Debian Package 설치하기
<https://docs.ros.org/en/jazzy/Installation/Ubuntu-Install-Debs.html>
- ROS2 Documentation, ROS2 환경 설정하기
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Configuring-ROS2-Environment.html>
- ROS2 Documentation, Jazzy Jalisco Release 정보
<https://docs.ros.org/en/jazzy/Releases/Release-Jazzy-Jalisco.html>
- ROS2 Documentation, ROS2 배포판과 지원 기간
<https://docs.ros.org/en/jazzy/Releases.html>
- ROS2 Documentation, 설치 문제 해결
<https://docs.ros.org/en/jazzy/How-To-Guides/Installation-Troubleshooting.html>

다음 장 예고

Ubuntu 24.04에 ROS2 Jazzy를 설치하고 첫 통신까지 검증했다. 4장에서는 Linux Terminal의 `pwd`, `ls`, `cd`, `mkdir`, `cp`, `mv`, `grep`, `find`를 실제 저장소에서 사용한다. Git으로 변경을 기록하고, WSL에 연결된 VS Code에서 소스 코드를 안전하게 편집하는 방법도 익힌다.

다음은 직접 만드는 시간

설치와 첫 통신을 마쳤다면, 직접 Node를 만들어 보세요.

4~6장 Terminal · Git · Turtlesim · Workspace

7~16장 Python Node · Topic · QoS · Service

17~26장 Action · Namespace · TF · URDF · RViz2

27장 Gazebo Sim에서 TurtleBot3로 벽 따라가기

GitHub에서 이어가기

README의 30분 Quick Start로 예제 코드를 실행하세요.

전체 목차와 전자책·종이책 안내는 BOOK.md에서 확인하세요.

무료 샘플이 도움이 됐다면 저장소 링크를 공유해 주세요.

<https://github.com/freshmea/ros2-maker-guide>